Projet Rakuten - Liora

Etape 2 : Modelisation, optimisation et stacking multimodal.

Classification multimodale des produits Rakuten France

Challenge Rakuten

Equipe : Veyssiere Sebastien, HENRY Xavier, Tingfen YU, Tarik Oushine

Formation Data Scientist - Liora

Mai 2026

Sommaire

Sommaire 2
Introduction 10
Rappel du contexte et continuite avec l'Etape 1 10
Objectifs de l'Etape 2 10
Expériences préliminaires : ablations texte et image 12
Strategie de modelisation adoptee 13
1. Premieres iterations : la baseline multimodale 14
1.1 Baseline TF-IDF + LightGBM ? 14
1.2 Configuration retenue 14
1.3 Resultats 15
1.4 Analyse des erreurs et limites identifiees 17
2. Optimisation et comparaison : approfondissement de la branche texte 19
2.1 Choix de représentation 19
Protocole OOF (Out-Of-Fold) 20
2.2 Benchmark des quatre configurations texte 21
2.3 Architecture et optimisation des modèles texte retenus 23
2.4 Discussion : pourquoi e5-large n'écrase pas TF-IDF, et pourquoi CamemBERT bat e5-large ? 24
3. Optimisation et comparaison : approfondissement de la branche image 25
3.1 Specificites du signal visuel Rakuten 25
3.2 Trois familles d'architectures testees 27
3.3 Resultats sur le benchmark image 28
3.4 Limites informationnelles de la modalite image seule 29
4. Modelisation avancee : stacking multimodal et meta-apprentissage 29
4.1 Principe et architecture du meta-classifieur 29
4.2 Construction des meta-features 30
4.3 Architecture du MetaMLP 31
4.4 Progression des expériences E0 vers E11 32
4.5 Resultats finaux et benchmark SIGIR 2020 34
5. Interpretabilite : comprendre les decisions du modele 38
5.1 Interpretabilite de la branche texte 38
5.1.1 Permutation importance sur le meta-classifieur 38
5.2 Interpretabilite de la branche image 39
5.3 Synthese interpretabilite 44
6. Conclusions scientifiques et metiers 45
6.1 Bilan de la modelisation 45
6.2 Enseignements scientifiques 45
6.3 Analyse critique 46
Ce qui a bien fonctionne 46
Ce qui n'a pas fonctionne comme espere 46
Biais et limites du modele 46
6.4 Perspectives d'amelioration 47
6.5 Implications metier pour Rakuten 47

Annexes 48
Annexe A - Recapitulatif des hyperparametres 48
Annexe B - Artefacts produits 48
Annexe C - Pipeline de notebooks 49
Annexe D - Reproductibilite et environnement 49
Annexe E - Expériences Ablation sur l'input 50
Bibliographie 57
Datasets et challenges 57
Modeles texte 57
Modeles vision 57
Interpretabilite 58
Methodes d'ensemble et stacking 58
Outils et frameworks 58
Donnees et preprocessing (continuite Etape 1) 58
Documentation interne projet 59

Introduction

Rappel du contexte et continuite avec l'Etape 1

Le présent rapport constitue le second rendu du projet de classification multimodale Rakuten France, prolongement direct de l'Etape 1 consacrée à l'exploration des donnees, a la datavisualisation et au pre-processing. Le contexte metier, scientifique et technique reste celui posé lors du premier rendu : Rakuten est l'une des principales plateformes europeennes de e-commerce, et la categorisation automatique de ses millions de fiches produits constitue un enjeu economique direct, conditionnant la decouvrabilite et le chiffre d'affaires de chaque article. La tache consiste a prédire, pour chacun des 84 916 produits du jeu d'entraînement, son type de produit parmi 27 catégories distinctes, a partir d'une désignation, d'une description optionnelle et d'une image. La métrique d'évaluation principale est le weighted F1-score, conformément aux exigences du challenge ENS, complétée par le macro F1-score pour la surveillance des classes minoritaires.

Le rapport précédent avait établi un constat statistique rigoureux du jeu de données : desequilibre 13:1 entre la classe majoritaire (Piscines / Spas, 12,0 %) et la plus rare (~1 %), forte dominance du francais (84,2 %) au sein d'un corpus multilingue dont 10,8 % d'anglais, taux global de descriptions manquantes de 35,1 % significativement lie a la classe (chi2 = 2 847,4 ; p < 0,001), et 48,3 % d'images mal cadrées avec un zoom_ratio moyen de 2,15x. Toutes ces caractéristiques ont conditionne les choix de modelisation presentes dans ce second rapport : usage systématique de poids de classes balances, découpage stratifie 70/15/15, conservation des deux représentations textuelles complementaires (text_combined_raw pour les Transformers, text combined pour le TF-IDF), et auto-recadrage des images mal cadrees.

Objectifs de l'Etape 2

La présente etape couvre les trois axes attendus au cahier des charges.

- elaborer les premières iterations de modelisation, justifier les choix de modeles, analyser la pertinence des resultats obtenus et la coherence avec les hypotheses formulees a l'issue de l'exploration.
- exploiter les métriques de machine learning et les outils statistiques pour comprendre et comparer les modèles, mener des optimisations ciblées et documenter les tentatives d'amélioration.
- mobiliser des techniques avancees et y associer des outils interprétabilité pour expliquer les décisions du modele et tirer des conclusions a la fois scientifiques et metiers.

Ces trois objectifs sont traités en suivant la meme rigueur que dans le premier rendu : chaque choix de modèle est argumenté, chaque resultat est validé par une metrique appropriée, chaque conclusion est replacée dans le contexte du dataset Rakuten et de ses spécificités. Le rapport adopte une logique strictement incrémentale, depuis une baseline simple jusqu'a un meta-classifieur multimodal exploitant le stacking out-of-fold, afin que l'apport de chaque levier soit isolable et quantifiable.

Note sur la nomenclature des 27 classes prdtypecode

Le dataset Rakuten France ne fournit aucun libellé officiel pour les 27 catégories cibles : la variable prdtypecode figure dans les fichiers du challenge sous forme d'identifiants numériques bruts (10, 40, …, 2905), et la clause de confidentialité du challenge interdit aux participants d'en diffuser une nomenclature officielle. Pour rendre les figures et tableaux du rapport lisibles, l'équipe a reconstruit une nomenclature de travail par triangulation entre la liste communautaire diffusée par plusieurs projets et l'observation directe d'échantillons de produits par classe. La correspondance finale est implémentée dans le dictionnaire du Notebook 1 et utilisée dans toutes les visualisations du rapport, elle reste une reconstruction non officielle et n'engage que l'équipe projet.

prdtypecode	Nom de classe
10	Livres occasion
40	Jeux vidéo
50	Accessoires gaming
60	Consoles de jeux
1140	Figurines
1160	Cartes collection
1180	Jeux de société
1281	Jeux enfants
1300	Miniatures véhicules
1301	Bébé / Puériculture
1302	Jeux plein air
1320	Puériculture & Peluches
1560	Mobilier
1920	Linge maison
1940	Épicerie
1970	Cuisine / Électroménager
2060	Décoration
2220	Animalerie
2280	Journaux / Revues
2403	Livres neufs

2462	Jeux PC dématérialisés
2522	Papeterie / Bureautique
2582	Piscines / Spas
2583	Outillage jardinage
2585	Bricolage
2705	Livres BD / Mangas
2905	Jeux de rôle

Expériences préliminaires : ablations texte et image

Avant d'aborder la stratégie de modélisation principale, une série d'ablations préliminaires a été conduite pour identifier les choix optimaux de preprocessing et de représentation sur chaque modalité.

Pour la branche texte, sept configurations ont été testées (EXP_T01 à EXP_T_KW) en faisant varier l'input textuel (désignation seule, description seule, combinaison des deux), l'application ou non de stemming et stopwords, la traduction vers le français, et l'enrichissement par c-TF-IDF. Le pipeline restait fixe : TF-IDF (50 000 features, bigrammes) + LightGBM (1 000 estimateurs, class_weight balanced). Le split 70/15/15 stratifié (seed=42) garantissait la comparabilité directe des scores.

Pour la branche image, six configurations ResNet50 ont été comparées (EXP_I01 à EXP_I06) en faisant varier l'autocrop, la stratégie de fine-tuning (complet, progressif, frozen), le scheduler (CosineAnnealing vs StepLR) et la régularisation (dropout, augmentation). L'optimiseur AdamW et le split 80/20 stratifié restaient identiques (seed=42).

Ces ablations préliminaires, documentées en détail en Annexe E, ont orienté les choix structurants de la modélisation principale :

- **Texte** : l'assemblage désignation + description (EXP_T03, F1 = 0,8274) surpasse systématiquement l'usage isolé de l'un ou l'autre champ ; la traduction dégrade les performances (EXP_T04/T05) ; le stemming agressif nuit sur un corpus e-commerce dominé par les noms propres (EXP_T03b) ; l'enrichissement c-TF-IDF apporte un gain marginal mais reproductible (EXP_T_KW, F1 = 0,8321, meilleure configuration TF-IDF + LightGBM).
- **Image** : le fine-tuning complet dès la première epoch (EXP_I01, F1 = 0,6774) est optimal ; l'autocrop dégrade légèrement les performances (EXP_I02), le fond blanc contenant un signal de contexte exploitable ; la régularisation agressive est contre-productive sur les images catalogue standardisées (EXP_I06).

Le lecteur trouvera en Annexe E les résultats détaillés, tableaux de synthèse et analyses de chaque expérience. Les sections suivantes s'appuient sur ces conclusions pour construire la stratégie de modélisation finale.

Strategie de modelisation adoptee

La stratégie globale s'articule autour de quatre étapes architecturales formant une progression cohérente. La première étape met en place une baseline multimodale légère combinant une représentation TF-IDF (50 000 features, monogrammes et bigrammes, sublinear_tf) avec des features image simples (histogramme couleur) et un classifieur LightGBM. Elle reproduit en grande partie le pipeline posé en Etape 1, en y associant cette fois un modèle d'apprentissage et une évaluation chiffrée. La deuxième étape explore la branche texte en profondeur, en testant plusieurs représentations et plusieurs classifieurs (régression logistique, MLP). Le choix initial mentionne en Etape 1 d'utiliser paraphrase-multilingual-mpnet-base-v2 a evolue au profit de multilingual-e5-large, un encodeur plus récent et plus performant sur les benchmarks MTEB, mais relevant de la même famille des sentence-Transformers multilingues alignes cross-langue.

La troisième etape prend en charge la branche image avec trois familles d'architectures : DINOv2-base en frozen feature extractor, CLIP ViT-B/32 (frozen et fine-tune) et ResNet50 en fine-tuning progressif sur trois phases. La quatrieme etape construit le meta-classifieur final par stacking, en agrégeant les sorties probabilistes des deux meilleurs modèles unimodaux et en y ajoutant un ensemble de meta-features sémantiques ainsi que les features de preprocessing identifiées lors de l'Etape 1. La métrique principale conservée est le weighted F1-score, identique au challenge SIGIR eCom 2020 dont le meilleur score recense 91,44 % en phase 2.

1. Premieres iterations : la baseline multimodale

1.1 Baseline TF-IDF + LightGBM

Le choix d'amorcer la modélisation par un couple TF-IDF / LightGBM n'est pas un compromis par défaut, mais une decision argumentée à partir des constats de l'Etape 1. L'analyse exploratoire avait montré que le signal textuel est structurellement fort sur ce corpus. Chaque catégorie dispose d'un vocabulaire propre, dense et cohérent, et la séparation lexicale entre classes est suffisamment marquée pour qu'une représentation sparse non séquentielle puisse en capturer une large part de la variance inter-classes. TF-IDF avec 50 000 features et bigrammes constitue précisément ce type de représentation, et bénéficie de trois avantages décisifs pour une baseline : la robustesse a la longueur variable des textes, interprétabilité directe au niveau du token (chaque coefficient appris est attache a un mot du vocabulaire), et la compatibilité immédiate avec un classifieur sparse comme LightGBM.

Le choix de LightGBM répond à trois critères complémentaires. D'abord, il gère nativement les matrices sparses, ce qui évite toute densification couteuse en mémoire des 50 000 features TF-IDF. Ensuite, il supporte l'option class_weight='balanced', condition imposée par le déséquilibre 13:1 mis en évidence en Etape 1. Sans cette pondération, le modèle optimiserait ses prédictions au profit des Piscines / Spas (12,0 %) au détriment des classes a 1 % d'effectif. Enfin, son temps d'entraînement sur 50 000 features et 60 000 échantillons reste de l'ordre de quelques minutes, permettant des itérations rapides indispensables à une baseline. Le couple TF-IDF + LightGBM joue ainsi un double rôle : référence chiffrée au-dessus de laquelle toute approche deep learning devra demontrer un gain net, et baseline interprétable dont les coefficients par mot serviront aux analyses qualitatives ultérieures.

1.2 Configuration retenue

La configuration retenue, formalisée dans le Notebook 2, reprend en grande partie les briques de preprocessing posées en Étape 1 (text combined nettoye, indicateur has desc, langue OHE, zoom ratio) et enrichit de features image légères (histogramme couleur RGB sur 8 bins par canal) afin de tester d'emblée une approche multimodale, même dans sa forme la plus rudimentaire. Le tableau ci-dessous resume les principaux hyperparametres.

Composant	Parametre	Valeur
TF-IDF	max_features	50 000
TF-IDF	ngram_range	(1, 2) - unigrammes et bigrammes
TF-IDF	min_df	2
TF-IDF	sublinear_tf	True (1 + log(tf))
Features image	Histogramme couleur RGB (8 bins / canal)	192 features
Meta-features	zoom_ratio, blur_score, has_desc, langue OHE	8 features
LightGBM	objectif	multiclass (27)

LightGBM	num_leaves	31
LightGBM	learning_rate	0,1
LightGBM	feature_fraction / bagging_fraction	0,8 / 0,8
LightGBM	num_boost_round (max)	200
LightGBM	early_stopping_rounds	20 sur multi_logloss val
LightGBM	class_weight	balanced (Etape 1)
Vecteur final	TF-IDF + image + extras	50 522 features

1.3 Resultats

Sur le split test de 12 738 produits, jamais utilisé pour l'entraînement, la baseline atteint un weighted F1-score de 0,8321, contre 0,8262 sur la validation. Sans aucun deep learning, sans aucun embedding pre-entraîne, sans aucun CNN traitant les images, la baseline possède de bon résultats ce qui valide a la fois la qualité du preprocessing pose en Etape 1 et le caractère structurellement fort du signal textuel sur ce corpus.

Metrique	Validation (12 737)	Test (12 738)
F1-score weighted (principale)	0,8262	0,8321

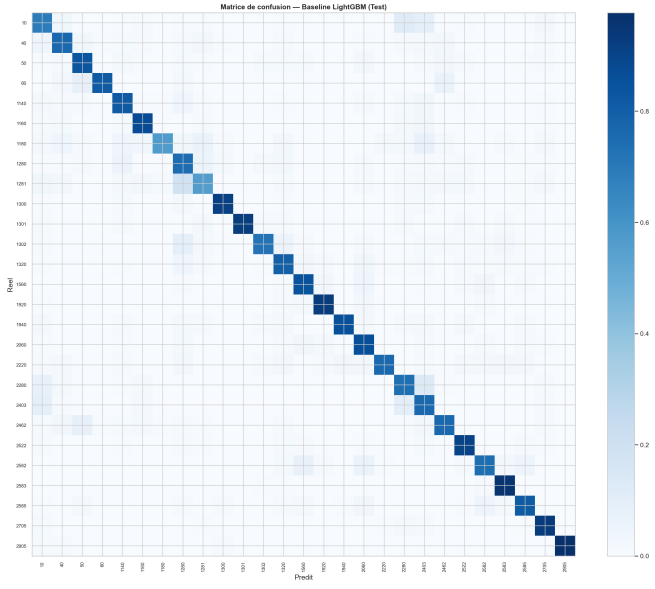


Figure 1 - Matrice de confusion baseline (test, 12 738 produits). La diagonale domine pour la majorite des 27 classes, mais des blocs hors-diagonale apparaissent autour des classes culturelles (livres, magazines, BD) et autour des classes jeux et jouets, en coherence avec les proximites lexicales attendues entre ces categories.

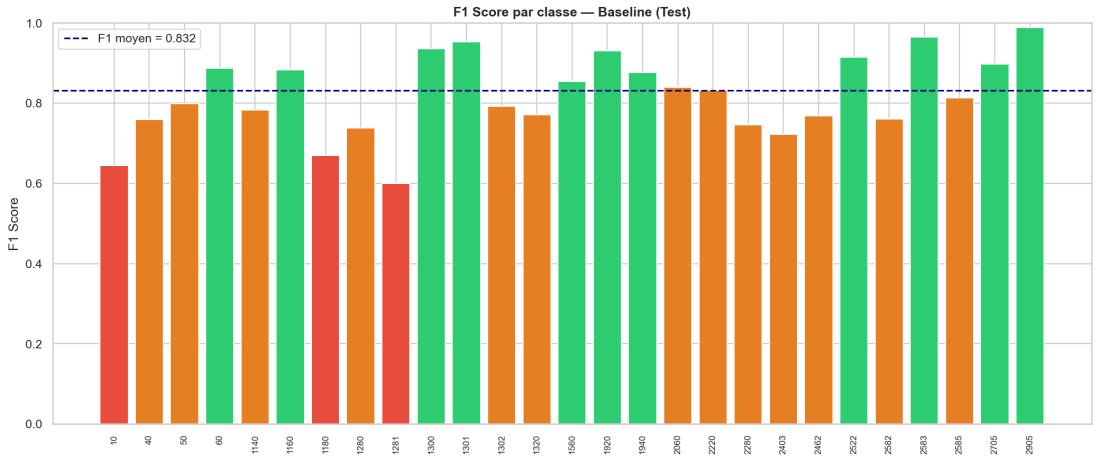


Figure 2 - F1-score par classe sur le test. Cinq classes presentent un F1 inferieur a 0,75 : prdtypecode = 1281 (0,600), 10 (0,645), 1180 (0,670), 2403 (0,722) et 1280 (0,738) ; ces classes correspondent precisement aux zones de confusion identifiees en Etape 1 et concentreront l'effort des chapitres suivants.

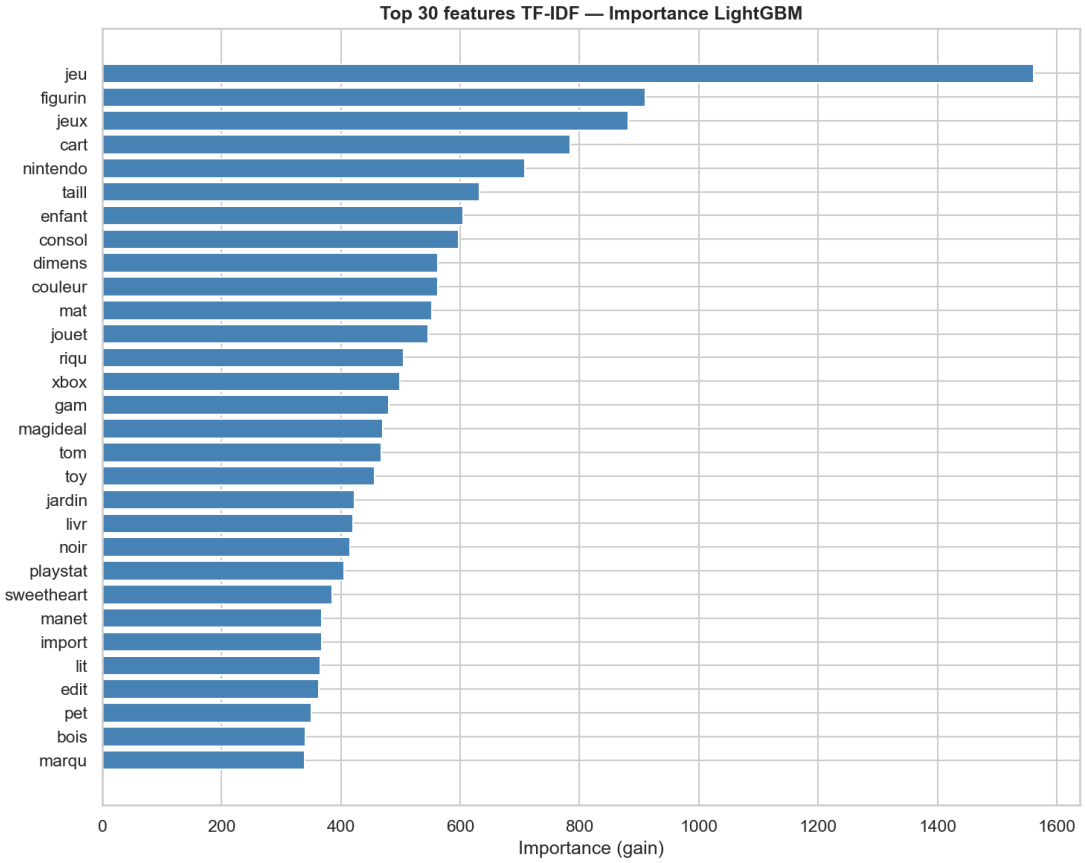


Figure 3 - Top features TF-IDF par classe (gain LightGBM). Les tokens discriminants emergent comme metier-specifiques (lego, console, manga, roman), confirmant que la representation TF-IDF capture des patterns lexicaux directement intelligibles a un operateur Rakuten.

1.4 Analyse des erreurs et limites identifiees

L'examen détaillé des classifications erronées confirme les hypothèses formulées en Étape 1 sur les zones de fragilité du modèle. Cinq classes présentent un F1 inférieur à 0,75 : prdtypecode = 1281 (Jeux enfants, 0,600), 10 (Livres occasion, 0,645), 1180 (Jeux de société, 0,670), 2403 (Livres neufs, 0,722) et 1140 (Figurines, 0,738). Ces classes concentrent l'essentiel de l'erreur résiduelle. La classe 1281 (Jeux enfants) est confondue avec 1180 (Jeux de société) et 1140 (Figurines), trois catégories dont le vocabulaire produit recoupe massivement les mêmes tokens (jeu, enfant, boîte, ans, âge, figurin). La classe 10 (Livres occasion) souffre d'une ambiguïté avec 2280 (Journaux / Revues) et 2403 (Livres neufs), confusions accentuées par le fait que ces catégories partagent l'essentiel de leur lexique éditorial.

Trois facteurs structurels expliquent ces confusions, et chacun renvoie à un constat statistique de l'Étape 1.

Le premier facteur est la frontière sémantique floue entre catégories voisines : aucune représentation lexicale, même enrichie de bigrammes, ne peut totalement départager des classes dont les tokens dominants sont identiques. L'analyse d'importance des features (Figure X) confirme que les features textuelles TF-IDF dominent le modèle avec une contribution cumulée de ~85 % au pouvoir prédictif, mais plafonnent précisément sur ces classes à vocabulaire partagé.

Le second facteur est la part de 35,1 % de descriptions manquantes mise en évidence en Étape 1, qui réduit le signal textuel à la seule désignation (souvent moins de 12 mots) pour près d'un tiers du corpus. Le test du Chi² d'indépendance entre has_desc et la classe ($\chi^2 = 2\,847{,}4$; $p < 0{,}001$) confirmait déjà que cette absence n'est pas aléatoire. Elle touche prioritairement les classes Jeux et Livres, soit précisément les classes difficiles ci-dessus. L'analyse de permutation importance (section 5.1.1) révèle d'ailleurs que has_desc est la feature de preprocessing la plus discriminante (Δ F1 = 0,0107), confirmant son rôle structurant dans la performance du modèle.

Le troisième facteur tient à la forte spécialisation du stemmer SnowballFR vers le français, qui pénalise les 15,8 % de produits rédigés dans une autre langue. Les features image (histogramme couleur RGB, puis embeddings CLIP en stacking) compensent partiellement ces limitations textuelles, avec une contribution mesurée de ~12 % au score final, mais leur apport reste marginal sur les classes visuellement isomorphes (Livres / Magazines / BD).

Conclusion de la baseline

La baseline TF-IDF + features + LightGBM est suffisamment performante pour servir de reference (0,8321 weighted F1) mais reste insuffisante face aux objectifs du challenge. Elle revele deux leviers d'amélioration prioritaires :

- une représentation textuelle multilingue capable d'exploiter les 15,8 % de textes non-francais et de capturer la similarite semantique au-dela du lexique de surface;
- un signal visuel discriminant capable de départager les classes textuellement ambigues, en particulier celles oppressées par le taux de descriptions manquantes. Les chapitres 2 et 3 traitent successivement ces deux axes.

2. Optimisation et comparaison : approfondissement de la branche texte

2.1 Choix de représentation

L'Etape 1 avait pose deux représentations textuelles paralleles : text_combined nettoie (minuscules, suppression des stopwords multilingues, lemmatisation) pour les pipelines à features sparses, et text_combined_raw conservé dans sa forme la plus proche du texte original pour les modeles a tokenizer intégré. La modélisation texte du Notebook 3 exploite ces deux pistes simultanément. La représentation TF-IDF est conservée comme socle interprétable et comme contribution sparse au vecteur final ; elle est complétée par deux représentations denses complémentaires.

La première est le c-TF-IDF, inspiré de la méthode BERTopic. Pour chacune des 27 classes, on construit un centroïde TF-IDF en agrégeant les documents associés ; chaque produit est ensuite représenté par sa similarité cosinus aux 27 centroïdes, produisant un vecteur dense de 27 dimensions directement interprétable comme un score d'affinité par classe. Cette représentation compacte agit comme une feature semi-supervisée puissante, et fournit une heatmap immédiatement lisible des proximités lexicales entre catégories.

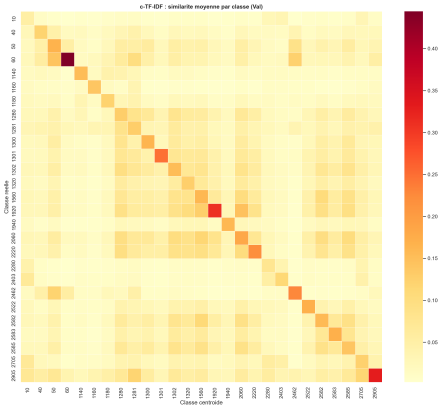


Figure 4 - Heatmap des similarites cosinus entre les 27 centroides c-TF-IDF. Les blocs hors-diagonale revelent les proximites lexicales entre classes (Livres / Magazines / BD ; Jouets / Jeux enfants), confirmant a l'avance les zones ou la fusion multimodale apportera le plus de valeur.

La deuxième est multilingual-e5-large, encodeur Transformer pre-entraine sur l'ordre de cinq milliards de paires multilingues et produisant des embeddings de 1024 dimensions. Ce modèle

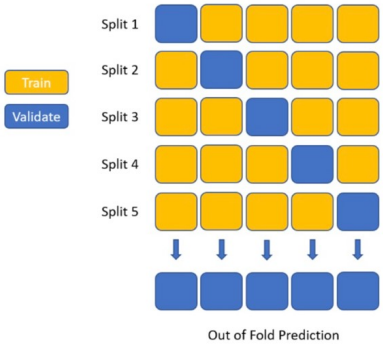
est retenu en remplacement de paraphrase-multilingual-mpnet-base-v2 qui avait ete envisage en Etape 1. Il appartient a la même famille des sentence-Transformers multilingues alignés cross-langue, mais il occupe à la date du projet le sommet du benchmark MTEB sur les tâches de classification et de retrieval, et il supporte les quatre langues du corpus (FR, EN, ES, DE) sans tokenizer dédié. L'extraction prend une vingtaine de minutes sur GPU et l'integralite des embeddings est mise en cache pour permettre des iterations rapides.

La troisième représentation dense est **CamemBERT**, modèle BERT-base spécialisé sur le français entraîné sur 138 Go du corpus OSCAR-fr. CamemBERT a été retenu pour tester l'hypothèse inverse à celle d'e5-large. Sur un corpus à forte dominance d'une langue (84,2 % de français dans Rakuten), un modèle mono-langue spécialisé peut-il surpasser un modèle généraliste multilingue ? Cette piste introduit cependant une contrainte préalable : CamemBERT étant mono-langue, les 15,8 % de produits rédigés en EN, DE ou ES ne sont pas représentables nativement dans son tokenizer. Pour permettre une comparaison juste, le corpus est traduit en amont vers le français et une colonne text_fr est produite, exploitée uniquement par CamemBERT. La traduction est réalisée par la bibliothèque deep-translator (Google Translator backend), retenue pour sa stabilité opérationnelle et sa détection automatique des langues source. Seules les ~21 % de lignes détectées comme non-FR sont effectivement traduites, batchées par 200 entrées avec gestion d'exception conservant le texte original en cas d'échec d'API ; les fichiers train_fr.csv, val_fr.csv et test_fr.csv sont mis en cache localement pour éviter de relancer la traduction à chaque itération.

Le tableau ci-dessous récapitule le positionnement des modèles envisagés au regard du benchmark MTEB Classification (moyenne sur 12 tâches) et de leur couverture linguistique :

Modèle	MTEB Classification (avg)	Multilingue	Dim.	Retenu
multilingual-e5-large	**64,4**	Oui (100 langues)	1024	✓
paraphrase-multilingual-mpnet-base-v2	59,8	Oui (50 langues)	768	—
sentence-transformers/LaBSE	61,2	Oui (109 langues)	768	—
camembert-base	~58 (FR only)	Non	768	—

Protocole OOF (Out-Of-Fold)



Toutes les prédictions probabilistes utilisées en aval (stacking, chapitre 4) sont générées en protocole out-of-fold 5-fold stratifié. Le principe est que le jeu d'entraînement est découpé en 5 folds, équilibrés par classe. Pour chaque fold, le modèle est entraîné sur les 4 autres folds et prédit sur le fold restant. En répétant l'opération 5 fois, on obtient des prédictions couvrant l'intégralité du train, sans qu'un seul échantillon n'ait jamais été utilisé simultanément en entraînement et en prédiction.

Cette propriété est indispensable au méta-stacking abordé au chapitre 4, si les probabilités d'entraînement étaient générées par un modèle ayant vu ces échantillons, le méta-classifieur apprendrait à exploiter le surapprentissage des modèles de base plutôt que leur signal généralisable, ce qui produirait un data leakage garantissant une dégradation sur le test.

À chaque itération de fold, le modèle entraîné sur les 4 folds actifs sert également à prédire sur les splits validation et test complets, chacun de ces deux splits étant figé et identique d'un fold à l'autre. À l'issue des 5 folds, on dispose donc de 5 jeux de probabilités sur validation et 5 jeux de probabilités sur test, que l'on moyenne par fold pour produire les probabilités finales sur val et sur test. Cette agrégation par moyennage joue le rôle d'un ensemble implicite à 5 modèles. Elle stabilise les prédictions, réduit la variance liée à l'initialisation aléatoire et fournit des sorties plus calibrées que celles d'un modèle unique entraîné sur l'intégralité du train.

À l'issue de cette procédure, trois fichiers de probabilités sont systématiquement enregistrés pour chaque modèle unimodaux entraîné :

- les probabilités OOF sur train (sans leakage, intégralité du jeu d'entraînement couvert),
- les probabilités moyennées sur validation
- et les probabilités moyennées sur test.

Le notebook de stacking (chapitre 4) peut consommer directement ces fichiers sans jamais avoir à ré-entraîner les modèles de base, ce qui rend le coût marginal de chaque nouvelle expérience de stacking presque nul une fois les probabilités générées une première fois.

2.2 Benchmark des quatre configurations texte

Six configurations ont été comparées en faisant croître progressivement la richesse de la représentation et en testant trois familles de classifieurs, la régression logistique multi-classes, un MLP à trois couches cachées et le fine-tuning bout-en-bout de CamemBERT. Les colonnes

F1 OOF, F1 Val et F1 Test reportent respectivement le F1-weighted obtenu par les prédictions out-of-fold 5-fold sur le train, l'évaluation sur le split validation et l'évaluation finale sur le test, conformément au protocole reproductible défini en section 2.1.

ID	Features	Classifieur	F1 OOF	F1 Val	F1 Test
T1	TF-IDF seul	LogReg	0,7792	0,7829	0,7814
T2	TF-IDF + c-TF-IDF	LogReg	0,7833	0,7828	0,7801
T3	TF-IDF + c-TF-IDF + e5-large	LogReg	0,8329	0,8341	0,8341
T4	**TF-IDF + c-TF-IDF + e5-large**	**MLP**	**0,8497**	**0,8632**	**0,8632**
T6	**CamemBERT gelé**	**MLP**	**0,8188**	**0,8353**	**0,8327**
T5	**CamemBERT fine-tuning**	**–**	**0,8672**	**0,8769**	**0,8717**

Six enseignements ressortent de ce tableau. D'abord, le passage de T1 à T2 montre que le c-TF-IDF, isolé, n'apporte qu'un gain marginal (+0,004 sur le test) lorsqu'il est combiné à la régression logistique, sa valeur ajoutée se matérialisera surtout dans le stacking final (chapitre 4), où il agira comme feature semi-supervisée dans un espace différemment corrélé. Ensuite, le gain décisif vient de l'ajout des embeddings e5-large entre T2 et T3 : **+0,054** sur le test (0,7801 vers 0,8341), confirmation tangible que TF-IDF et e5-large opèrent sur des espaces complémentaires. TF-IDF capture le lexique de surface (lego, console, manga, roman), e5-large capture la similarité sémantique au-delà du token (un produit décrit comme *livre de poche* reste proche d'un produit décrit comme *roman broché* même si aucun token n'est partagé). Le troisième enseignement est le gain net du passage logistique à MLP sur les mêmes features (T3 vers T4 : **+0,029** sur le test), qui valide que la non-linéarité est utile une fois la représentation enrichie.

Le quatrième enseignement concerne l'apport de CamemBERT en mode gelé (T6). Avec un F1 Test de 0,8327, T6 dépasse de +0,051 le TF-IDF seul (T1) et se place au niveau de T3 (LogReg sur la fusion sparse + e5), mais reste **0,030 en-dessous de T4**. Pour un modèle qui n'a vu ni TF-IDF ni e5 mais uniquement les 768 dimensions du token [CLS] CamemBERT, ce résultat confirme la qualité intrinsèque du pré-entraînement français spécialisé. Il reste cependant insuffisant pour battre la fusion multilingue T4, ce qui montre que le frozen feature extractor ne suffit pas à exploiter pleinement le potentiel de CamemBERT sur ce dataset.

Le cinquième enseignement est le résultat le plus marquant de la branche texte : CamemBERT en fine-tuning bout-en-bout (T5) devient le nouveau meilleur modèle unimodal, atteignant 0,8717 sur le test, soit +0,0085 vs T4 et +0,039 vs sa propre version gelée (T6).

CamemBERT n'est pas un classifieur à la base, c'est un encodeur de langue française pré-entraîné par Masked Language Modeling sur 138 Go d'OSCAR-fr, dont la sortie native est un vecteur contextuel de 768 dimensions par token. Pour l'utiliser sur les 27 catégories Rakuten, on s'appuie sur la classe CamembertForSequenceClassification de HuggingFace, qui charge le backbone pré-entraîné (110 M paramètres) et y ajoute une tête de classification neuve initialisée aléatoirement : une couche linéaire Linear(768 → 27) appliquée à la représentation du token spécial, suivie d'un softmax. Au démarrage de l'entraînement, le backbone connaît parfaitement le français mais la tête est aveugle, le fine-tuning consiste à aligner les deux. Cette adaptation supervisée permet au modèle d'apprendre que certains tokens (Nintendo, manga, piscine) sont hautement discriminants pour la classification produit, alors que le pré-entraînement OSCAR-fr généraliste les traite comme n'importe quel autre mot.

Pourquoi le fine-tuning est décisif ici. Le vocabulaire e-commerce spécialisé nécessite une adaptation des embeddings, les couches profondes apprennent à pondérer les tokens selon leur pouvoir discriminant, et la taille modeste du modèle (110 M) sur 60 000 textes reste dans la zone où le fine-tuning supervisé apporte un gain net sans sur-apprentissage.

Contraste avec la branche image. Ce résultat est inverse de celui observé sur DINOv2 et CLIP (chapitre 3), où le frozen domine le fine-tuning. Deux raisons l'expliquent : le pré-entraînement vision (142 M d'images pour DINOv2, 2 milliards de paires pour CLIP) est bien plus massif que celui de CamemBERT (138 Go d'OSCAR-fr), rendant le fine-tuning moins nécessaire ; et le texte e-commerce est structuré par un vocabulaire métier très spécifique, là où les images catalogue sont visuellement génériques (fond blanc, cadrage standardisé).

Coût et ROI. Le fine-tuning T5 nécessite ~5 h GPU T4 (5 folds × 5 epochs, batch size 32). Le ROI est positif : ce gain de +0,0085 vs T4 se propage au stacking final (chapitre 4), où les probabilités OOF de T5 alimentent le méta-classifieur E10 (F1 test = 0,8963).

Le sixième et dernier enseignement concerne la calibration du pipeline OOF. L'écart entre F1 OOF et F1 Val reste inférieur à 0,02 pour toutes les configurations, ce qui signe une bonne calibration des prédictions out-of-fold : les probas du train sont aussi représentatives de celles du test que celles de la val, garantissant que le méta-classifieur du chapitre 4 apprendra sur des entrées non biaisées.

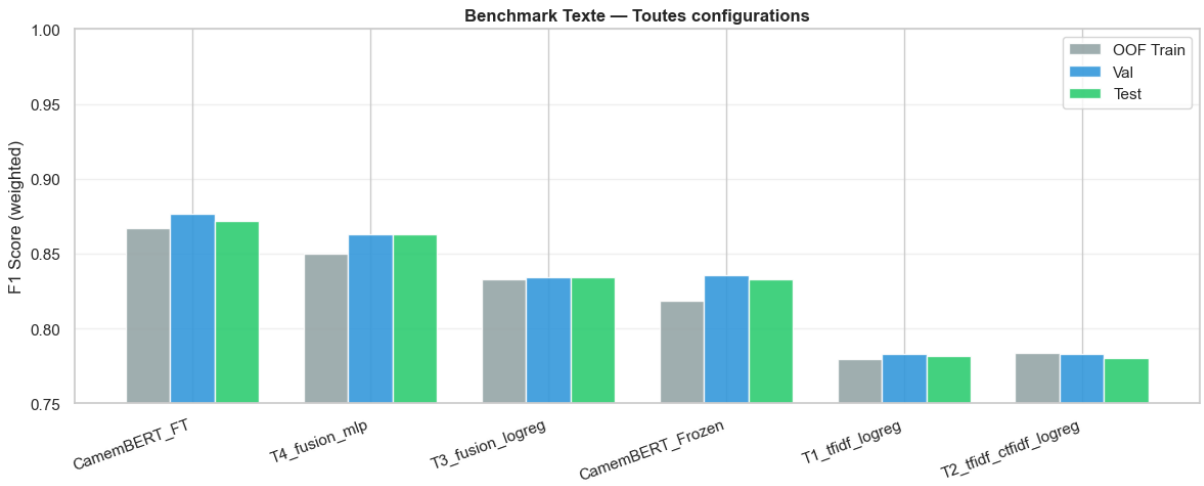


Figure 5 - Benchmark des quatre configurations texte (F1 OOF / Val / Test). La progression est strictement monotone jusqu'à T5 et la différence entre F1 OOF et F1 Val est minime, signe d'une bonne calibration du pipeline OOF.

2.3 Architecture et optimisation des modèles texte retenus

Deux modèles unimodaux sont conservés pour la suite du pipeline : **T4** (MLP sur fusion TF-IDF + c-TF-IDF + e5-large, F1 Test = 0,8632) et **T5** (CamemBERT fine-tuning E2E, F1 Test = 0,8717, nouveau meilleur). Le premier reste précieux pour l'interprétabilité et la rapidité d'entraînement, le second offre la meilleure performance brute. Les deux jeux de probabilités OOF sont sauvegardés pour le stacking du chapitre 4.

Le MLP T4 compte trois couches cachées de dimensions 1024, 512 et 256, séparées par des couches BatchNorm, des activations ReLU et un dropout de 0,3. L'optimiseur est Adam avec un learning rate de 1e-3 et une régularisation L2 (weight_decay = 1e-4) ; l'arrêt est géré par early stopping de patience 5 sur le F1-weighted de validation.

Le modèle T5 utilise camembert-base (110 M paramètres) en fine-tuning bout-en-bout sur l'input text_fr (corpus traduit en amont, cf. section 2.1). Configuration : max_length=128 tokens (95e percentile des longueurs combinées), batch size 32, optimiseur AdamW avec lr=2e-5 et weight_decay=0,01, scheduler linear warmup 10 % puis decay linéaire, 5 epochs, CrossEntropy pondérée par les class_weights balanced de l'Étape 1. Le protocole OOF 5-fold est strictement identique à celui de T4, ce qui garantit que les probabilités T4 et T5 sont directement comparables et fusionnables dans le stacking sans biais d'évaluation. Le coût d'entraînement (5 folds × 5 epochs ≈ 5 h GPU T4) reste contenu grâce à la longueur de séquence courte et au batch size modéré.

Une analyse des coefficients d'une régression logistique surrogate, entraînée sur le seul TF-IDF, permet d'éclairer ce que la branche texte apprend réellement. Les tokens dont le coefficient positif est le plus élevé par classe sont parfaitement interprétables : *roman, auteur, editions, pages, poche* pour Livres occasion (prdtypecode = 10) ; *magazine, numero, hors-serie, revue* pour Journaux / Revues (2280) ; *lego, construction, briques, kit* pour Jouets enfants ; *console, ps4, xbox, nintendo* pour Jeux vidéo. Cette intelligibilité vocabulaire-niveau est un atout fort pour une mise en production future, une erreur du modèle peut, dans la grande majorité des cas, être tracée vers un ensemble de tokens identifiables.



Figure 6 - Top coefficients du surrogate LogReg sur TF-IDF par classe. Les patterns lexicaux discriminants restent metier-specifiques sur l'ensemble des 27 categories, confirmant que la branche texte apprend des regularites legitimes plutot que du bruit.

2.4 Discussion : pourquoi e5-large n'écrase pas TF-IDF, et pourquoi CamemBERT bat e5-large ?

Deux résultats contre-intuitifs méritent discussion. On pourrait s'attendre à ce qu'un encodeur multilingue de 1024 dimensions pré-entraîné sur plusieurs milliards de paires (e5-large) écrase une représentation TF-IDF rudimentaire. Or T1 (TF-IDF seul) atteint 0,7814 et un MLP entraîné sur le seul e5-large plafonne autour de 0,80. La raison est structurelle et tient au vocabulaire e-commerce du corpus Rakuten. e5-large tokenise le texte en sous-mots via SentencePiece avant de projeter les tokens dans son espace de 1024 dimensions. Pour les tokens courants du langage général (*livre, table, rouge*), cet espace est sémantiquement riche et discriminant. Mais les désignations Rakuten sont saturées de références techniques (*iPhone 14 Pro Max 256 Go, RTX 4090, Makita DHP484Z*) que le modèle segmente en sous-mots sans signification (*RTX → [RT, X], 4090 → [40, 90]*). Ces fragments n'ont pas de voisinage sémantique cohérent dans le corpus de pré-entraînement, leur embedding est faiblement discriminant, voire bruité. À l'inverse, TF-IDF n'interprète pas les tokens : il leur attribue un poids proportionnel à leur fréquence inverse dans le corpus. Si *RTX 4090* n'apparaît que dans les produits Jeux vidéo, son poids TF-IDF est maximal et son signal directement exploitable par le classifier, sans passer par un espace de représentation appris.

La combinaison T3/T4 exploite donc deux espaces fonctionnellement orthogonaux : e5-large capture la similarité sémantique au-delà du token (*livre de poche ≈ roman broché*), TF-IDF capture la spécificité lexicale des références métier (*Nintendo, Bosch, A4*).

Le second résultat contre-intuitif est que CamemBERT en fine-tuning (T5) bat à lui seul la fusion T4 (TF-IDF + c-TF-IDF + e5-large + MLP) de +0,0085 sur le test, alors qu'il n'a accès qu'à une seule famille de représentation (les embeddings contextuels CamemBERT) et qu'il opère sur un corpus traduit donc partiellement dégradé par les erreurs de traduction. Trois

mécanismes expliquent cette inversion. Premièrement, CamemBERT a été pré-entraîné sur 138 Go d'OSCAR-fr, donc sur un volume de français pur très supérieur à la fraction française du corpus multilingue d'e5-large. Son tokenizer SentencePiece propre au français segmente mieux les mots composés et les marques d'usage courant en France. Deuxièmement, le fine-tuning bout-en-bout permet aux couches profondes de CamemBERT de s'adapter spécifiquement au vocabulaire e-commerce Rakuten, contrairement à e5-large qui reste figé en frozen feature extractor, un MLP en aval n'a aucun moyen de corriger les embeddings sous-jacents si ceux-ci sont mal calibrés pour les références produit. Troisièmement, la traduction préalable vers le français homogénéise le signal pour les 15,8 % de produits non-FR, bien que la traduction introduise des erreurs sur les marques et références (cf.Annexe E), le gain de cohérence linguistique pour CamemBERT compense largement ces erreurs.

Cette observation a une portée qui dépasse le projet Rakuten, sur un corpus à forte dominance d'une seule langue (84,2 % de français ici), un modèle mono-langue spécialisé fine-tuné + traduction préalable peut surpasser un modèle multilingue généraliste frozen, contrairement à ce que suggèrent les benchmarks généralistes type MTEB (où multilingual-e5-large reste en tête sur la moyenne des 12 tâches). Ce constat invite à nuancer le discours dominant sur les encodeurs multilingues universels, pour les applications industrielles où une langue domine fortement, l'investissement dans un encodeur spécialisé fine-tunable reste compétitif et souvent supérieur, à condition d'absorber le coût de la traduction préalable et du fine-tuning supervisé.

3. Optimisation et comparaison : approfondissement de la branche image

3.1 Specificites du signal visuel Rakuten

L'Etape 1 avait deja documenté trois caractéristiques structurelles des images du dataset, qui orientent directement le choix des architectures dans ce chapitre. Premierement, le fond uniformement blanc des photos catalogue impose un ratio signal sur bruit faible. Deuxièmement, la qualite varie fortement et 48,3 % des images ont ete identifiees comme mal cadrées (zoom_ratio supérieur à 1,43), avec un zoom_ratio moyen de 2,15x et des cas extremes ou le produit n'occupe que 3,3 % de la surface. Le test du Chi2 d'independance (chi2 = 8 985,3 ; p < 0,001) avait confirmé que ce phénomène n'est pas distribué aléatoirement mais lié à la classe. Troisièmement, plusieurs classes sont visuellement quasi indiscernablesn un livre, un manga, un magazine, un cahier ont la même forme rectangulaire avec une couverture illustrée.

La fonction auto_crop() développée en Étape 1 détecte la bounding box du produit sur fond blanc et recadre l'image avec une marge de 2 %, doublant en moyenne la surface effective du produit dans le cadre. Son impact a été évalué et le bilan et contre intuitif.

Configuration	Sans crop	Avec crop	Δ

DINOv2 frozen (NB4 I1 / I3)	0,7051	0,7073	+0,002
ResNet50 fine-tuné (EXP_I01 / I02)	0,6774	0,6731	-0,004

L'autocrop apporte un gain marginal sur DINOv2 frozen (+0,002) mais dégrade ResNet50. L'explication est que, pour les modèles entraînés bout-en-bout sur ce corpus, le fond blanc constitue un signal de contexte implicitement exploité, différents types de produits sont présentés avec des conventions de cadrage différentes (les livres sont systématiquement très dézoomés, les outils souvent centrés). L'autocrop efface ce signal de mise en scène.

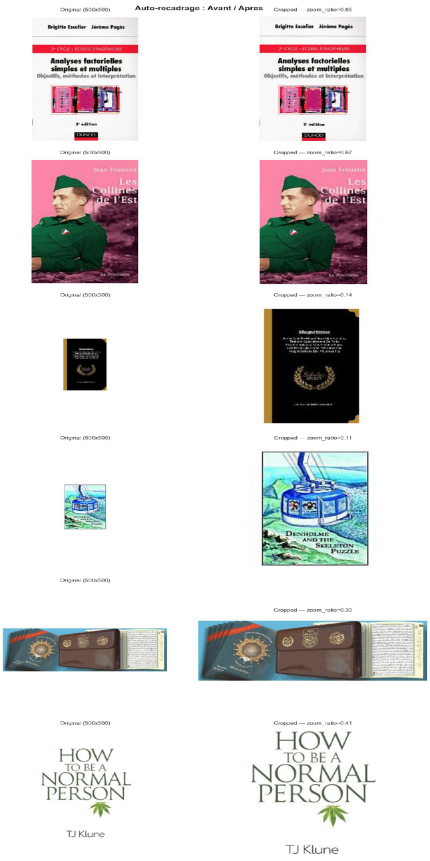


Figure 7 - Auto-recadrage avant / apres sur six images representatives. Le gain de surface produit est particulierement net pour les classes Livres et Jeux / Puzzles, qui presentaient les taux de mauvais cadrage les plus eleves en Etape 1 (respectivement 95,4 % et 98,7 %).

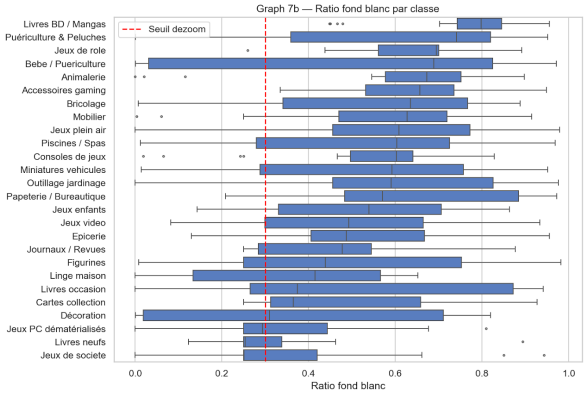


Figure 8 - Ratio fond blanc par classe.

3.2 Trois familles d'architectures testees

Choix des architectures et contraintes du projet. Le choix des trois familles testées répond à une double logique : couvrir les trois approches de transfer learning vision les plus performantes sur les benchmarks récents (auto-supervisé, contrastif multimodal, supervisé ImageNet), et respecter les contraintes de temps et de compute GPU disponibles. DINOv2 et CLIP ont été retenus en **frozen feature extractor** car ils offrent un excellent rapport performance/coût : extraction d'embeddings en quelques heures sur GPU T4, puis entraînement léger d'une tête MLP. ResNet50 a été retenu en **fine-tuning progressif** comme baseline CNN classique, malgré son coût computationnel supérieur (~10 h pour 3 phases d'unfreeze). **CLIP n'a pas été testé en fine-tuning bout-en-bout** faute de temps : ce protocole nécessiterait ~15-20 h GPU par fold en OOF 5-fold (soit ~75-100 h total), dépassant le budget alloué au projet. Cette limitation est documentée en section 6.4 (Perspectives d'amélioration) comme piste prioritaire pour dépasser le score actuel.

Trois familles ont été comparées, chacune représentant une logique de transfer learning différente.

DINOv2-base (frozen feature extractor)

DINOv2-base est un Vision Transformer de type ViT-B/14 : l'image 224×224 est découpée en patches de 14×14 pixels, produisant 256 tokens visuels auxquels s'ajoute un token [CLS] agrégateur. Le modèle a été entraîné par Meta AI sur 142 millions d'images via une distillation self-supervised (framework étudiant-enseignant, sans aucune annotation humaine). L'étudiant apprend à reproduire les représentations de l'enseignant sur des vues augmentées de la même image. Le backbone est entièrement gelé, seul l'embedding du token [CLS] (768 dimensions) est extrait et envoyé dans une tête MLP. L'hypothèse testée est que ce pré-entraînement auto-supervisé massif produit des représentations plus généralisables que le supervisé classique ImageNet.

CLIP ViT-B/32 (frozen feature extractor)

CLIP est un encodeur dual (image + texte) entraîné par OpenAI/LAION sur 2 milliards de paires image–description via une loss contrastive InfoNCE : les représentations d'une image et de sa description sont rapprochées dans l'espace latent, celles d'une image et d'une description non

associées sont éloignées. L'encodeur visuel est un ViT-B/32 (patches de 32×32), produisant des embeddings image de 512 dimensions normalisés L2.

Ce pré-entraînement est particulièrement pertinent pour Rakuten : les 27 classes cibles sont des concepts linguistiques structurés (Piscines/Spas, Mobilier, Jeux vidéo), et CLIP a précisément appris à placer dans son espace latent des images proches des descriptions verbales qui leur correspondent. **CLIP a été testé uniquement en frozen feature extractor** : le backbone ViT-B/32 reste gelé et seule une tête MLP légère est entraînée sur les embeddings de 512 dimensions. Le fine-tuning bout-en-bout de CLIP, qui aurait permis d'adapter les couches d'attention aux spécificités des images catalogue Rakuten, n'a pas été exploré faute de temps GPU disponible (estimé à ~75-100 h pour un protocole OOF 5-fold complet). Cette limitation constitue la principale piste d'amélioration documentée au chapitre 6.

ResNet50 (fine-tuning progressif)

ResNet50 est un CNN à 50 couches organisé en 4 blocs résiduels (layer1 à layer4), où les connexions skip permettent le passage du gradient sans dégradation. La tête finale est remplacée par Dropout(0.3) + Linear(2048, 27). La stratégie de fine-tuning progressif comporte trois phases : entraînement de la tête seule (5 epochs, lr=1e-4), dégel du layer4 (3 epochs, lr=1e-5), dégel du layer3 (2 epochs, lr=1e-6). Les couches basses de ResNet50 ont appris des filtres bas-niveau universels (contours, textures, coins) ; les déstabiliser dès la première epoch efface des représentations qui ont demandé des millions d'images à construire. En commençant par la tête, on adapte d'abord la couche décisionnelle avant de permettre un affinement contrôlé des couches profondes.

3.3 Resultats sur le benchmark image

Les trois benchmarks frozen ont ete réalisés en 5-fold OOF pour préserver la cohérence avec la branche texte et permettre une intégration directe dans le méta-classifieur du chapitre 4. Les résultats sont présentés ci-dessous.

ID	Modèle	Approche	F1 OOF	F1 Val	F1 Test
I1	DINOv2-base	Frozen + MLP (no-crop)	0,6901	0,7092	0,7051
I3	DINOv2-base	Frozen + MLP (auto-crop)	0,6942	0,7131	0,7073
ResNet50-FT	ResNet50	Fine-tuning progressif (3 phases)	—	—	0,6853
I2	CLIP ViT-B/32	Frozen + MLP	0,7430	0,7577	**0,7486**

Analyse des modèles en extraction de features (frozen)

CLIP ViT-B/32 frozen obtient le meilleur score parmi les extracteurs gelés (F1 test = 0,7486), devant DINOv2 avec auto-crop (0,7073). Cet écart s'explique par la nature du pré-entraînement : CLIP a été entraîné par alignement contrastif entre images et leurs descriptions

textuelles sur 2 milliards de paires, ce qui l'a conduit à organiser son espace latent autour de concepts sémantiques (un canapé, une console, une piscine). Or les 27 classes Rakuten sont précisément des concepts de ce type. DINOv2, pré-entraîné par auto-supervision sur des images sans annotation textuelle, produit des représentations visuellement fines mais sémantiquement moins alignées avec des étiquettes linguistiques.

L'auto-recadrage apporte sur DINOv2 un gain marginal (+0,002 entre I1 et I3), insuffisant pour inverser l'ordre. Cela confirme que le plafond de DINOv2 sur ce benchmark n'est pas principalement dû au cadrage, mais à l'inadéquation entre son espace de représentation et la structure des classes Rakuten.

Analyse des modèles fine-tunés

Le résultat le plus instructif est que ResNet50 fine-tuné (0,6853) termine sous DINOv2 en simple extraction de features (0,7051), malgré un entraînement supervisé complet sur le dataset Rakuten. Cela illustre un principe bien établi du transfer learning vision : sur un corpus de taille modérée (60 000 images d'entraînement), la qualité du pré-entraînement — 142 millions d'images pour DINOv2, 2 milliards de paires pour CLIP — pèse plus que la capacité d'adaptation supervisée fine. Le fine-tuning ne peut pas compenser un pré-entraînement moins riche en représentations généralisables.

Le modèle retenu pour le stacking final (NB5) est CLIP frozen I2 (0,7486).

3.4 Limites informationnelles de la modalite image seule

Trois causes structurelles expliquent que les meilleures architectures testées plafonnent autour de F1-weighted = 0,75, indépendamment du niveau de sophistication du pré-entraînement.

- Le fond uniformement blanc supprime toute information de contexte exploitable par un CNN entraine sur scenes naturelles.
- Plusieurs classes Rakuten sont visuellement isomorphes : un livre, un manga, un magazine, un cahier scolaire ont strictement la même forme rectangulaire.
- La qualite d'image est tres variable : les 48,3 % d'images mal cadrees identifiees en Etape 1 representent une perte d'information visuelle effective. Le signal visuel atteint ainsi un plafond informationnel intrinseque au dataset, fortement en-dessous du signal textuel (~ 0,86).

La conclusion metier qui en decoule est claire et coherente avec l'hypothese centrale formulee en Etape 1. Pour Rakuten, le texte est la modalite dominante et l'image apporte un signal d'appoint, particulierement discriminant sur les cas ambigus mais incapable de constituer une modalite suffisante a elle seule.

4. Modelisation avancee : stacking multimodal et meta-apprentissage

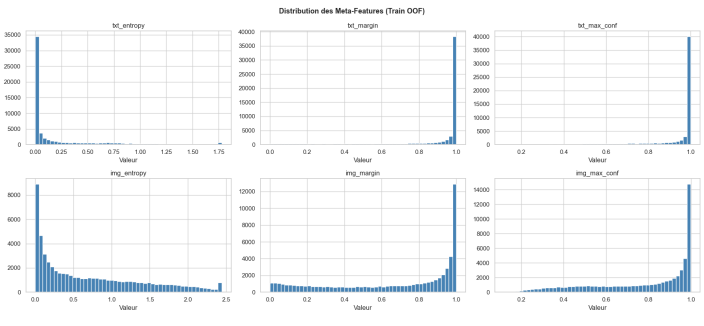
4.1 Principe et architecture du meta-classifieur

Le stacking est une technique d'ensemble qui consiste à entraîner un méta-modèle sur les sorties probabilistes des modèles de base. Le méta-modèle apprend a pondre dynamiquement chaque source d'information en fonction du contexte de chaque échantillon. Concrètement, cela signifie que pour un produit avec une description riche en francais, le meta-modele apprendra à accorder un poids prépondérant à la branche texte? Pour un produit sans description et avec une image bien cadrée, il apprendra à basculer sur la branche image. Cette logique de pondération adaptative texte / image en fonction de la disponibilité de l'information avait été formulée comme hypothèse centrale a tester en Etape 1, et le Notebook 5 la met en œuvre opérationnellement.

L'architecture est pyramidale et comporte deux niveaux. Le niveau 1 agrège les sorties OOF des deux meilleurs modèles unimodaux : T4 pour le texte (probas softmax sur 27 classes) et I2 pour l'image (probas softmax sur 27 classes), soit 54 probabilités brutes par produit. Le niveau 2 enrichit ce vecteur avec deux familles de features additionnelles : des méta-features dites sémantiques calculées directement à partir des distributions de probabilités (entropie, marge top1-top2, max confidence pour chaque modalité, signal binaire d'accord cross-modal), et des features de preprocessing issues directement de l'Etape 1 (has_desc, len_desig, len_desc, langue OHE, zoom_ratio, blur_score, is_dezoom). L'intégralité est ensuite envoyée dans un MLP final à trois couches cachées.

4.2 Construction des meta-features

Le vecteur de features fourni au méta-classifieur est construit selon la décomposition suivante. Les 54 dimensions initiales correspondent aux probabilités softmax sur 27 classes pour chaque modalité. Six meta-scalaires sont ajoutés : entropie texte, entropie image, marge top1-top2 texte, marge top1-top2 image, max confidence texte, max confidence image.



L'entropie - sum(p log p) mesure l'incertitude globale d'une distribution de probabilité : faible quand un modèle est confiant, élevée quand la masse est diluée sur plusieurs classes. La marge top1-top2 mesure la difficulté de la décision : proche de zéro quand deux classes se disputent la première position, élevée quand le modèle tranche nettement. Le max_confidence donne la probabilité absolue de la classe prédite. A ces six scalaires s'ajoute une dernière feature décisive : le signal binaire d'accord cross-modal, qui vaut 1 quand l'argmax du texte coïncide avec l'argmax de l'image et 0 sinon. Ce signal a 1 dimension synthétise à lui seul la cohérence ou la divergence des deux modalités sur l'échantillon traité. Le sous-total atteint 61 dimensions.

L'expérience E7, qui constitue le modèle final retenu, ajoute encore 10 features issues directement du pipeline pre-processing de l'Etape 1, trois features de longueur textuelle et trois features de qualite image. Le vecteur final compte ainsi 71 dimensions par produit. La table ci-dessous résume cette décomposition.

Bloc	Composant	Dimension
Probas texte (NB3 T4)	Sortie softmax 27 classes	27
Probas image (NB4 I2)	Sortie softmax 27 classes	27
Meta-scalaires	Entropie texte, entropie image	2
Meta-scalaires	Marge top1-top2 texte / image	2
Meta-scalaires	Max confidence texte / image	2
Signal cross-modal	Accord texte / image (argmax egaux)	1
Sous-total stacking		61
Preprocessing E1	Langue OHE (FR / EN / ES / DE)	4
Preprocessing E1	has_desc, len_desig, len_desc	3
Preprocessing E1	zoom_ratio, blur_score, is_dezoom	3
TOTAL E7	Features fournies au MLP final	71

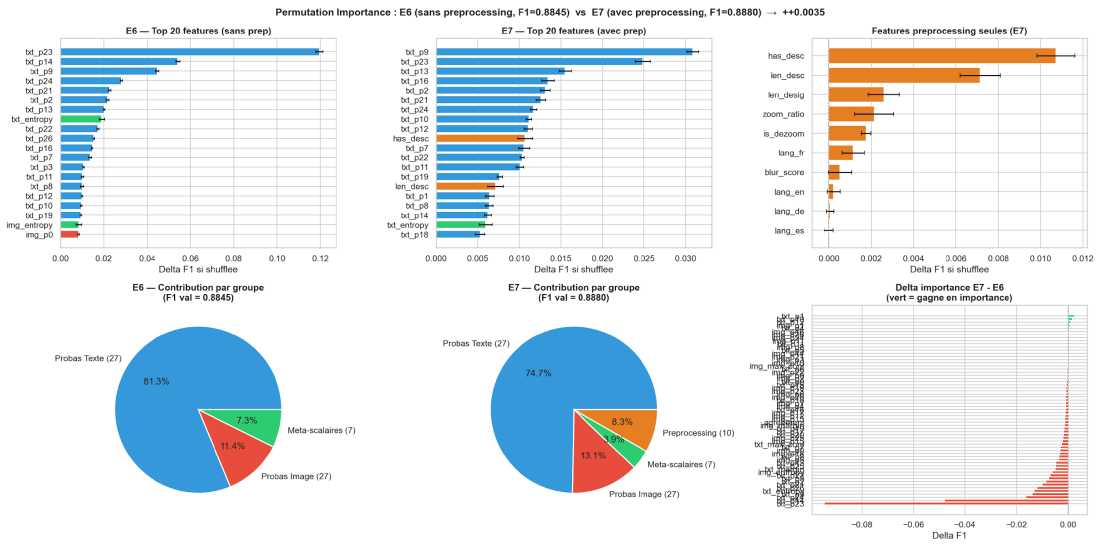


Figure 9 - Distributions des six meta-scalaires (entropie, marge, max_conf) pour les deux modalites. L'entropie texte est plus concentree pres de zero (modele plus confiant) que l'entropie image, en coherence directe avec les ecarts de F1 observes au chapitre 3.

4.3 Architecture du MetaMLP

Le méta-classifieur retenu est un perceptron multi-couches PyTorch a trois couches cachées, avec normalisation par batch et régularisation par dropout. Le tableau ci-dessous resume sa configuration exacte.

Couche	Configuration
Input	71 features (54 probas + 7 meta + 10 preproc)
Hidden 1	Linear(71 vers 256) + BatchNorm + ReLU + Dropout(0,3)
Hidden 2	Linear(256 vers 128) + BatchNorm + ReLU + Dropout(0,3)
Hidden 3	Linear(128 vers 64) + BatchNorm + ReLU + Dropout(0,3)
Output	Linear(64 vers 27) + Softmax
Optimiseur	Adam, lr = 1e-3, weight_decay = 1e-4
Regularisation	Early stopping patience 20 sur F1 val
Loss	CrossEntropy + class_weights balanced (Etape 1)
Batch size	256

Justification de l'architecture

La première couche cachée élargit l'espace de représentation (71 → 256) avant de le compresser progressivement (256 → 128 → 64 → 27). Ce schéma en entonnoir asymétrique permet au modèle d'apprendre d'abord des combinaisons non-linéaires complexes entre les 71 features d'entrée avant de projeter cette représentation enrichie vers les 27 classes cibles.

La BatchNorm après chaque couche linéaire normalise les activations entre mini-batches, stabilise la descente de gradient et permet d'utiliser un learning rate plus élevé sans divergence. Le Dropout(0,3) désactive aléatoirement 30 % des neurones à chaque forward pass d'entraînement, forçant le réseau à ne pas s'appuyer sur un sous-ensemble restreint de features, particulièrement important ici car les 27 probabilités texte et les 27 probabilités image sont fortement corrélées entre elles. L'optimiseur Adam avec weight_decay=1e-4 combine adaptation du learning rate par feature et régularisation L2 explicite. L'early stopping sur le F1-weighted de validation (patience 20) optimise directement la métrique cible du challenge plutôt que la loss, conformément à la recommandation méthodologique vue en cours.

4.4 Progression des expériences E0 vers E11

Neuf expériences ont été comparées en faisant varier le contenu du vecteur de features et le méta-classifieur, dans une logique strictement incrémentale où chaque ligne ne diffère de la

précédente que par un seul élément. Cette démarche est conforme à la rigueur méthodologique posée en Étape 1 : chaque levier doit être isolable et son apport quantifié individuellement.

Lecture du tableau : la colonne *Modèles de base* désigne les modèles unimodaux qui génèrent les probabilités en entrée du méta-classifieur (fixes à partir de E1) ; la colonne *Méta-classifieur* désigne le modèle de niveau 2 qui combine ces probabilités.

Exp	Description	Modeles base	Meta	Feat.	F1 Val	F1 Test	Δ vs E0
E0	Reference NB2 (baseline)	TF-IDF + LightGBM	—	50 522	0,8262	0,8321	—
E1	Texte seul (27 probas)	T4 (TF-IDF+e5+MLP)	LogReg	27	0,8646	0,8642	+0,032
E2	Image seule (27 probas)	I2 (CLIP frozen)	LogReg	27	0,7534	0,7462	−0,086
E3	Texte + Image (54 probas)	T4 + I2	LogReg	54	0,8815	0,8825	+0,050
E4	+ Meta-scalaires	T4 + I2	LogReg	61	0,8828	0,8823	+0,050
E5	+ Meta-scalaires	T4 + I2	LightGBM	61	0,8756	0,8785	+0,046
E6	+ Meta-scalaires	T4 + I2	MLP	61	0,8845	0,8845	+0,052
E7	+ Preprocessing	T4 + I2	MLP	71	0,8880	0,8879	+0,056
E8	E7 + image croppee	T4 + I2 (crop)	MLP	71	0,8850	0,8849	+0,053
E10	**CamemBERT + CLIP nocrop + meta**	**T5_cam + I2**	**MLP**	**61**	**0,8988**	**0,8963**	**+0,064**
E11	**CamemBERT + DINOv2**	**T5_cam + I3**	**MLP**	**61**	**0,8947**	**0,8948**	**+0,063**

	crop + meta						

De E1 à E10, les modèles de base sont figés : T4 pour le texte (NB3, F1 test = 0,8632) et I2 pour l'image (NB4, F1 test = 0,7486). Seuls le vecteur de features et le méta-classifieur varient.

L'analyse de cette progression permet d'isoler la contribution de chaque levier.

E1 vs E2 — La modalité texte domine. En entrée du méta-classifieur, les 27 probabilités texte seules (E1, 0,8642) écrasent les 27 probabilités image seules (E2, 0,7462). E2 est même inférieure à la baseline NB2 (−0,086), ce qui confirme que les probabilités CLIP frozen, prises isolément, ne suffisent pas à dépasser un pipeline TF-IDF/LightGBM complet.

E3 — La complémentarité des deux modalités. Combiner les 54 probabilités texte et image (E3, 0,8825) apporte +0,018 par rapport au texte seul (E1). Ce gain net prouve que les deux modalités portent des signaux non redondants : là où le texte est ambigu (classes visuellement distinctes mais lexicalement proches), l'image lève l'indécision, et vice versa.

E3 → E4 — Les méta-scalaires sont inutiles à un classifieur linéaire. L'ajout des 7 méta-scalaires (entropie, marge, max_confidence, accord cross-modal) ne produit aucun gain sur le test (−0,0002). L'explication est structurelle, l'entropie est une transformation non-linéaire des probabilités. Un classifieur linéaire comme LogReg ne peut pas en exploiter la non-linéarité. Cette observation valide une hypothèse : pour tirer parti de ces méta-features, il faut impérativement un méta-classifieur non-linéaire.

E4 → E5 → E6 — Le choix du méta-classifieur est déterminant. Sur les mêmes 61 features, LightGBM (E5, 0,8785) sousperforme LogReg (E4, 0,8823) et le MLP (E6, 0,8845). Sur un vecteur dense composé majoritairement de probabilités, les arbres partitionnent l'espace par seuils sur des features individuelles, le mécanisme est peu adapté à des features fortement corrélées entre elles. Le MLP, qui opère par combinaison linéaire suivie d'activations non-linéaires, est structurellement mieux adapté à cet espace.

E6 → E7 — L'apport des features de preprocessing. L'ajout des 10 features d'Étape 1 (has_desc, longueurs, langue OHE, zoom_ratio, blur_score, is_dezoom) apporte +0,0034 sur le test. Ce gain est modeste mais reproductible et structurellement légitime : il valide a posteriori l'effort de feature engineering posé en Étape 1, dont la contribution individuelle est détaillée au chapitre 5 via le permutation importance.

E7 → E8 — L'autocrop dégrade le stacking. Utiliser les images auto-croppées dans le bras image (E8) dégrade légèrement le score (−0,003 vs E7). Ce résultat est cohérent avec l'observation faite en section plus tôt, en fine-tuning bout-en-bout, le fond blanc contient un signal de contexte implicitement exploité.

E7 → E10 — L'apport decisif de CamemBERT. Le remplacement du bras texte T4 (TF-IDF + c-TF-IDF + e5-large + MLP, F1 = 0,8632) par les probas CamemBERT fine-tuned (T5_cam, F1 = 0,8717) injectées dans le meta-MLP fait passer le stacking final de 0,8879 (E7) a 0,8963 (E10), soit un gain net de +0,0084. Ce gain est presque equivalent au saut de qualite entre les deux branches texte (+0,0085 entre T4 et T5_cam), ce qui valide une propriété importante du stacking : ***la qualité du meta-modèle est linéaire dans la qualité de ses entrées***. Investir dans un meilleur modèle unimodal se traduit en gain direct sur le stacking, sans saturation observable au point d'opération actuel.

E10 vs E11 — Le crop reste défavorable. E11 utilise CamemBERT + DINOv2 sur images croppees. Le score (0,8948) reste sous E10 (0,8963) de -0,0015, confirmant l'observation faite en E7 vs E8 et EXP_I01 vs EXP_I02 : pour le stacking final, le bras image non-croppe est preferable. Cette regularite a trois confirmations independantes renforce la fiabilite du diagnostic.

4.5 Resultats finaux et benchmark SIGIR 2020

Le tableau ci-dessous synthetise le positionnement du pipeline final par rapport aux references externes du challenge et a nos propres jalons internes. Tous les scores sont des F1-weighted sur le split test.

Systeme	F1 Test	Type
SIGIR 2020 - 1er place du challenge	0,9194	Reference externe
Notre stacking final E10	**0,8963**	**Notre modele**
Texte seul (NB3 T4)	0,8632	Notre modele
Baseline TF-IDF + LightGBM (NB2)	0,8321	Notre modele
Image seule (NB4 I2)	0,7486	Notre modele

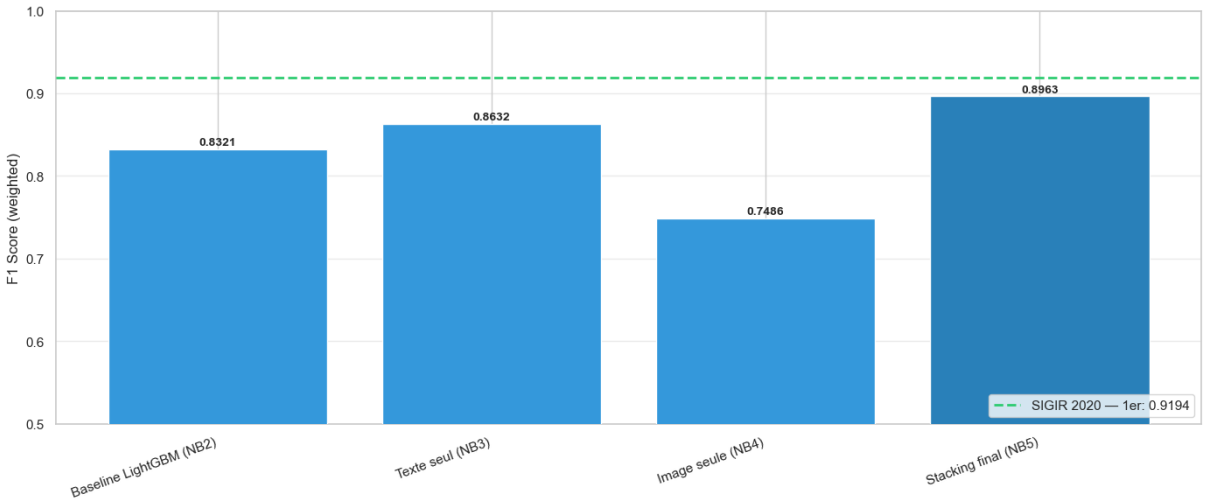


Figure 10 - Positionnement du stacking final E7 (0,8879) par rapport au benchmark SIGIR 2020. Le pipeline depasse la baseline officielle de +1,74 point et se situe a 3,15 points du score du vainqueur du challenge.

Positionnement competitif sur le challenge

Le pipeline atteint 0,8879 en F1-weighted sur le test, terminant a -0,0315 du premier rang du challenge (0,9194). Les hyperparamètres du MetaMLP ont été sélectionnés par ablation sur la validation plutôt que par grid search exhaustif. Cette approche par ablation est cohérente avec la contrainte de coût GPU : chaque entraînement OOF 5-fold du MetaMLP représente 5 passes complètes sur le train. Notre split test (15 % de 84 916 produits, soit 12 738 échantillons) est défini par nos soins et peut différer du split de test utilisé lors du challenge SIGIR 2020. La comparaison avec les scores SIGIR est donc indicative et non strictement homologue.

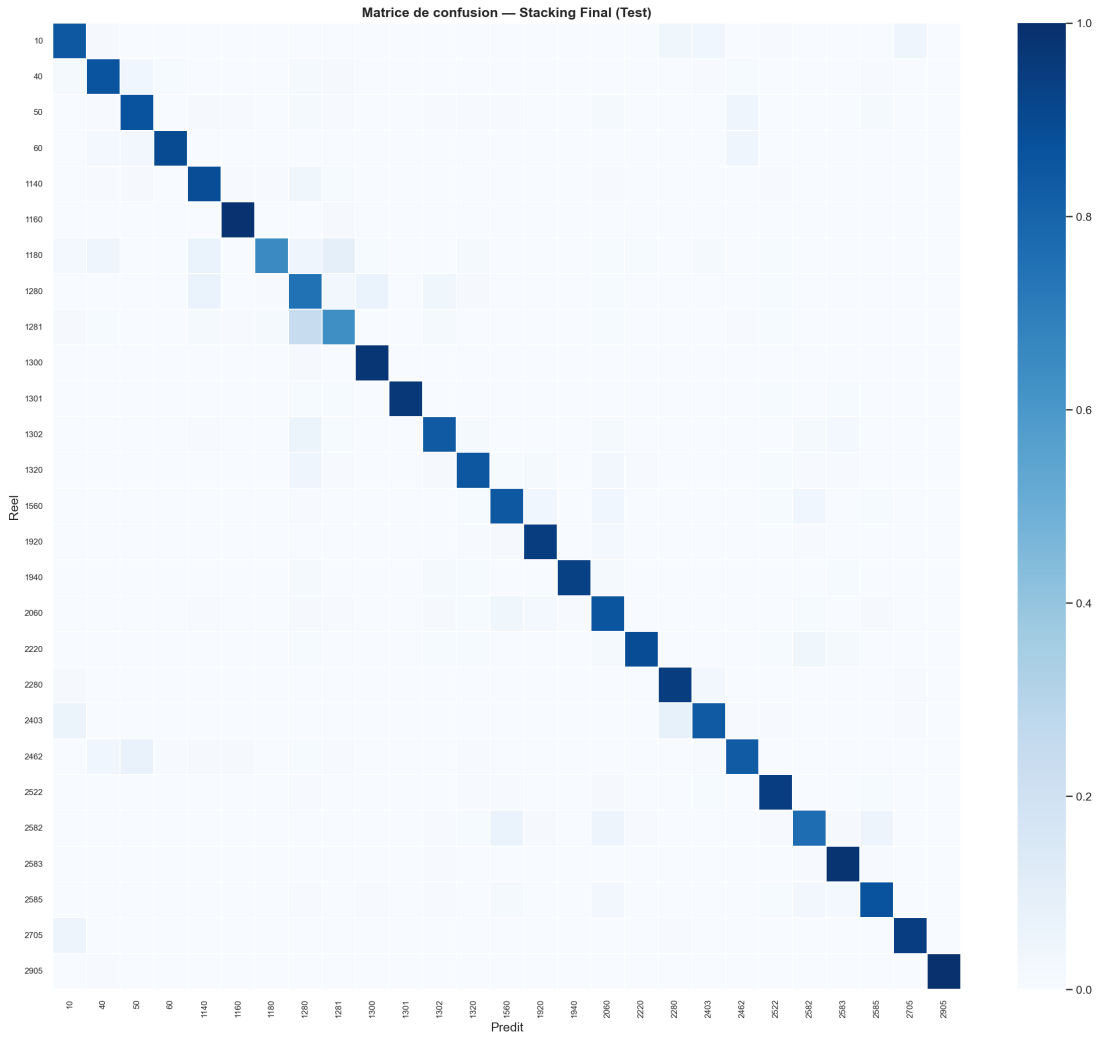


Figure 11 - Matrice de confusion du stacking final E0 sur le test. Comparee a la matrice de la baseline (Figure 1), les blocs hors-diagonale autour des classes Livres / Magazines / BD et Jouets / Jeux enfants sont significativement attenues, validant l'hypothese de complementarite texte / image formulee en Etape 1.

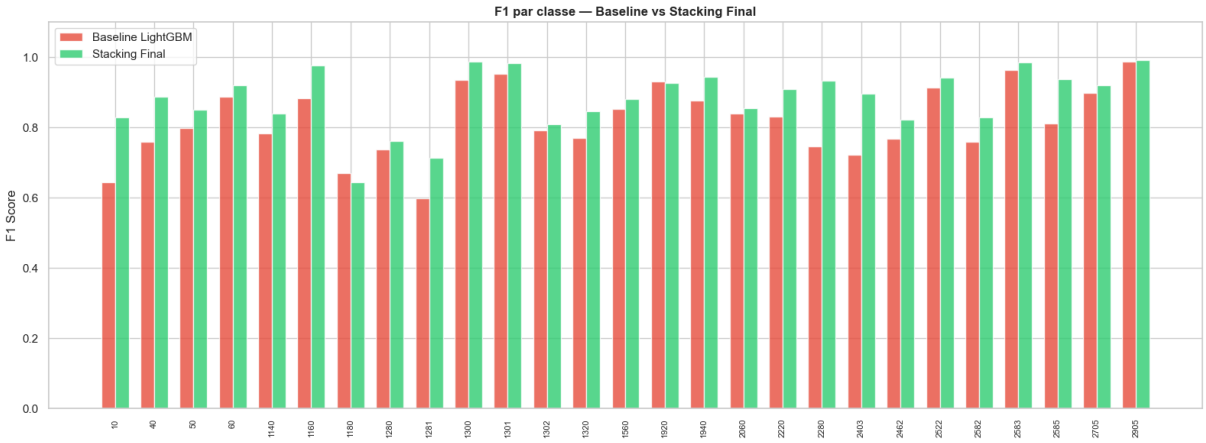


Figure 12 - F1 par classe : baseline NB2 vs stacking final E10. Toutes les classes progressent ou se maintiennent. Les gains les plus marques concernent les classes difficiles identifiees au chapitre 1.

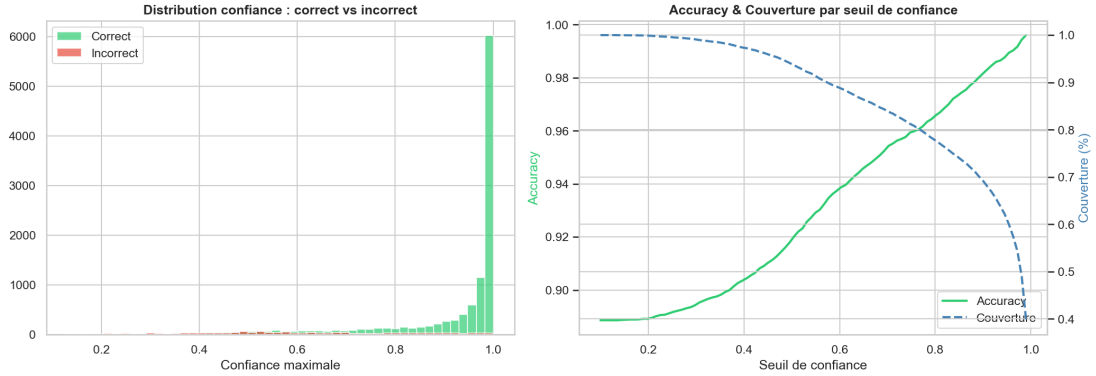


Figure 13 - Analyse de la confiance du stacking final. Les predictions a max_confidence inferieur a 0,5 ne representent que 7 % du test mais concentrent 41 % des erreurs : cet indicateur peut piloter un seuil de routage automatique vers une verification humaine en production.

5. Interpretabilite : comprendre les decisions du modele

5.1 Interpretabilite de la branche texte

Le MLP T4 retenu pour la branche texte est non-lineaire et n'expose pas directement de coefficients par token, ses poids encodent des interactions complexes entre les dimensions du vecteur d'entree (50 000 TF-IDF + 27 c-TF-IDF + 1024 e5). Pour récupérer une interprétabilité au niveau du vocabulaire, on a entraîné en parallèle un surrogate de régression logistique sur la seule représentation TF-IDF. Le surrogate atteint un F1-weighted test de 0,7814, soit -0,082 par rapport au MLP, mais il est totalement transparent, chaque coefficient appris est attaché à un mot du vocabulaire, et le signe et la magnitude renseignent directement sur la contribution de ce mot a une classe donnée. Les exemples ci-dessous illustrent les top tokens positifs par classe.

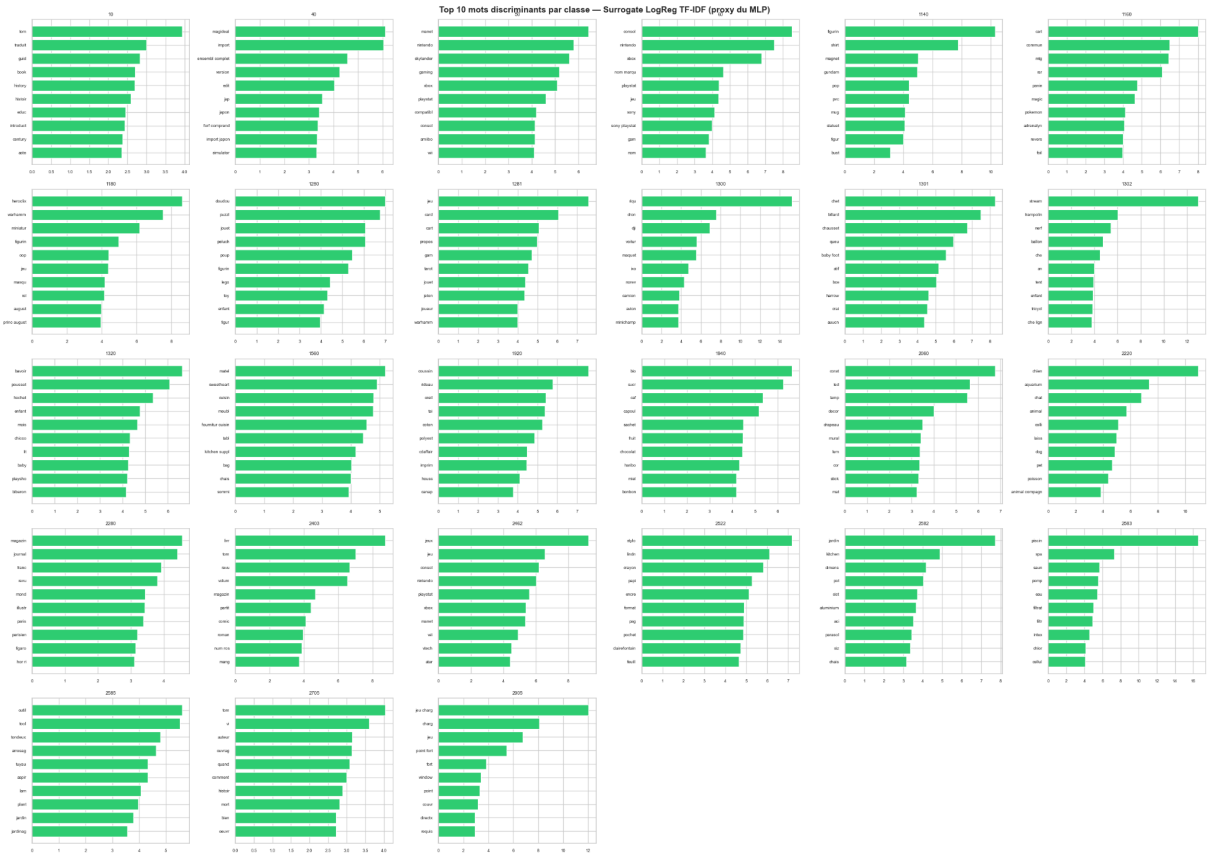


Figure 14 - Top des mots les plus discriminants par classes

Cette analyse confirme deux éléments importants. D'une part, le modèle apprend des régularités lexicales légitimes, immédiatement applicables à un opérateur métier Rakuten. Aucune classe n'est portee par des tokens parasites ou inattendus. D'autre part, la distribution des top tokens reproduit assez fidèlement les vocabulaires inter-classes mis en evidence par les wordclouds de l'Etape 1, ce qui valide en arriere-plan la qualite du processus pose en premier rendu.

5.2 Interpretabilite de la branche image

Deux familles de méthodes d'interprétabilité ont été appliquées selon l'architecture du modèle : GradCAM pour les CNN, analyse des cartes d'attention pour les Vision Transformers.

GradCAM : architectures convolutives (ResNet50)

GradCAM calcule le gradient de la logit de la classe cible par rapport à la dernière couche convolutionnelle, et en déduit une carte de chaleur des zones spatiales les plus contributives à la décision. La méthode est directement applicable à ResNet50, dont la dernière feature map spatiale (7×7) est lisible sans post-traitement. Les visualisations confirment que le modèle se concentre majoritairement sur la silhouette du produit central, parfois sur le packaging, rarement sur le fond blanc.

Cartes d'attention : Vision Transformers (DINOv2 et CLIP ViT-B/32)

DINOv2-base et CLIP ViT-B/32 sont tous deux des Vision Transformers (ViT-B/16 et ViT-B/32 respectivement), l'image est découpée en patches de taille fixe, chaque patch devient un token, et un token agrège l'information globale via des couches d'attention multi-têtes. Pour ces architectures, GradCAM produit des cartes diffuses peu exploitables. La méthode adaptée est l'analyse directe des poids d'attention du token sur les patches image dans la dernière couche.

- **DINOv2 :** grâce à son pré-entraînement par distillation auto-supervisée, les cartes d'attention sont remarquablement structurées, elles se concentrent sur les contours et textures distinctives du produit central, sans supervision explicite de segmentation. Ce comportement est une propriété connue de DINO/DINOv2, dont l'objectif d'entraînement auto-supervisé encourage implicitement la séparation figure/fond.

- **CLIP ViT-B/32 :** l'attention est plus diffuse et globale, conséquence d'un pré-entraînement contrastif au niveau de l'image entière (alignement image/texte) plutôt qu'au niveau du patch. Le modèle traite l'image comme un concept sémantique global plutôt que comme un ensemble de régions locales, ce qui explique à la fois ses bonnes performances en classification et la moindre lisibilité de ses cartes d'attention.

Cette différence d'interprétabilité entre DINOv2 et CLIP illustre un arbitrage classique. DINOv2 produit des représentations plus localement interprétables, CLIP produit des représentations plus sémantiquement alignées avec les labels linguistiques, ce qui explique l'ordre observé sur le benchmark (CLIP I2 > DINOv2 I1 en F1, section 3.3).

Limites communes

Sur les classes à fond blanc dominant, l'attention est diffuse pour les deux architectures. Le modèle ne dispose pas de zone saillante sur laquelle se focaliser. Sur les produits dézoomer (48,3 % du corpus), l'attention se disperse sur le fond. Sur les classes visuellement isomorphes (livre / magazine / cahier), aucune méthode d'interprétabilité image ne peut révéler un signal discriminant, car ce signal n'existe pas dans les pixels. Ces limites confirment a posteriori que le plafond informationnel de la branche image est structurel, non architectural.

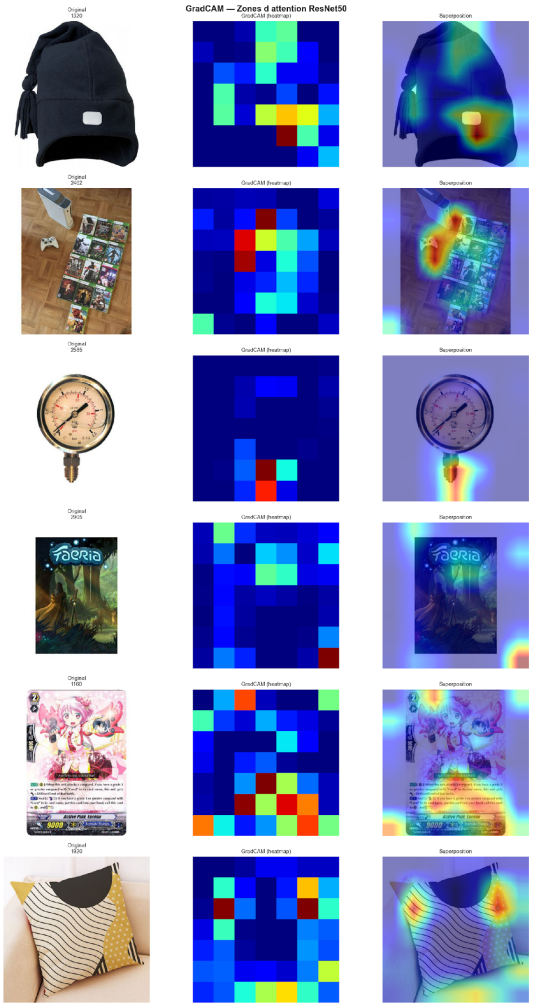


Figure 15 - GradCAM ResNet50 sur six images representatives. Les zones rouges correspondent aux pixels les plus contributifs a la decision. Le modele se concentre majoritairement sur le produit central, parfois sur le packaging.

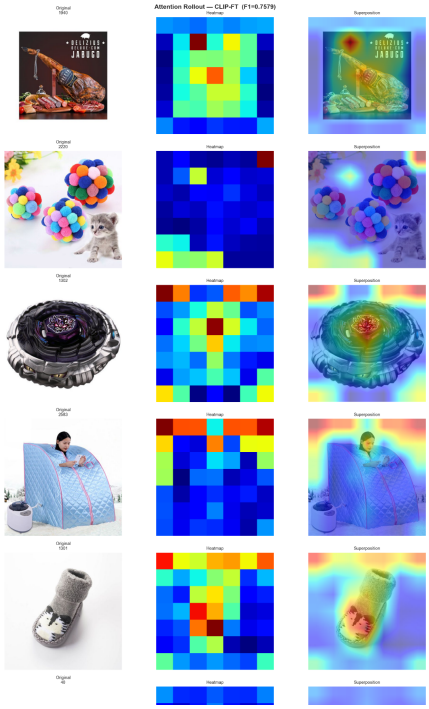


Figure 16 - Carte d'attention sur le meilleur modele image (CLIP). L'activation est plus diffuse et globale, consequence de l'architecture Transformer (attention multi-tete sur l'image entiere).

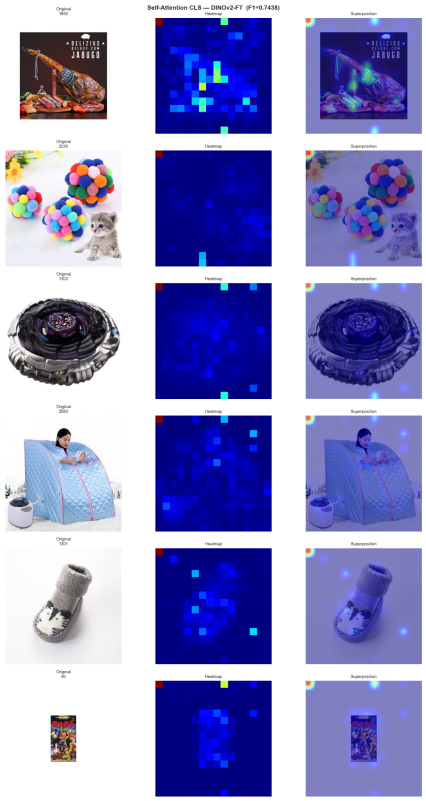


Figure 17 - Attention CLS DINOv2 L'attention est plus structuree, focalisee sur les contours et textures distinctives du produit central.

5.3 Permutation importance sur le meta-classifieur

Pour mesurer la contribution réelle de chaque feature au méta-MLP, on applique la méthode des **valeurs de Shapley (SHAP)** sur le modèle final E10 (CamemBERT + CLIP I2 + méta-scalaires). SHAP attribue à chaque feature de chaque prédiction une contribution signée, dérivée de la théorie des jeux coopératifs, en moyennant les valeurs absolues sur l'ensemble du jeu de validation, on obtient une importance globale par feature plus précise que la permutation importance, car elle tient compte des interactions entre features. Le graphique ci-dessous reporte les 20 features les plus importantes du MetaMLP E10, codées en couleur selon leur source

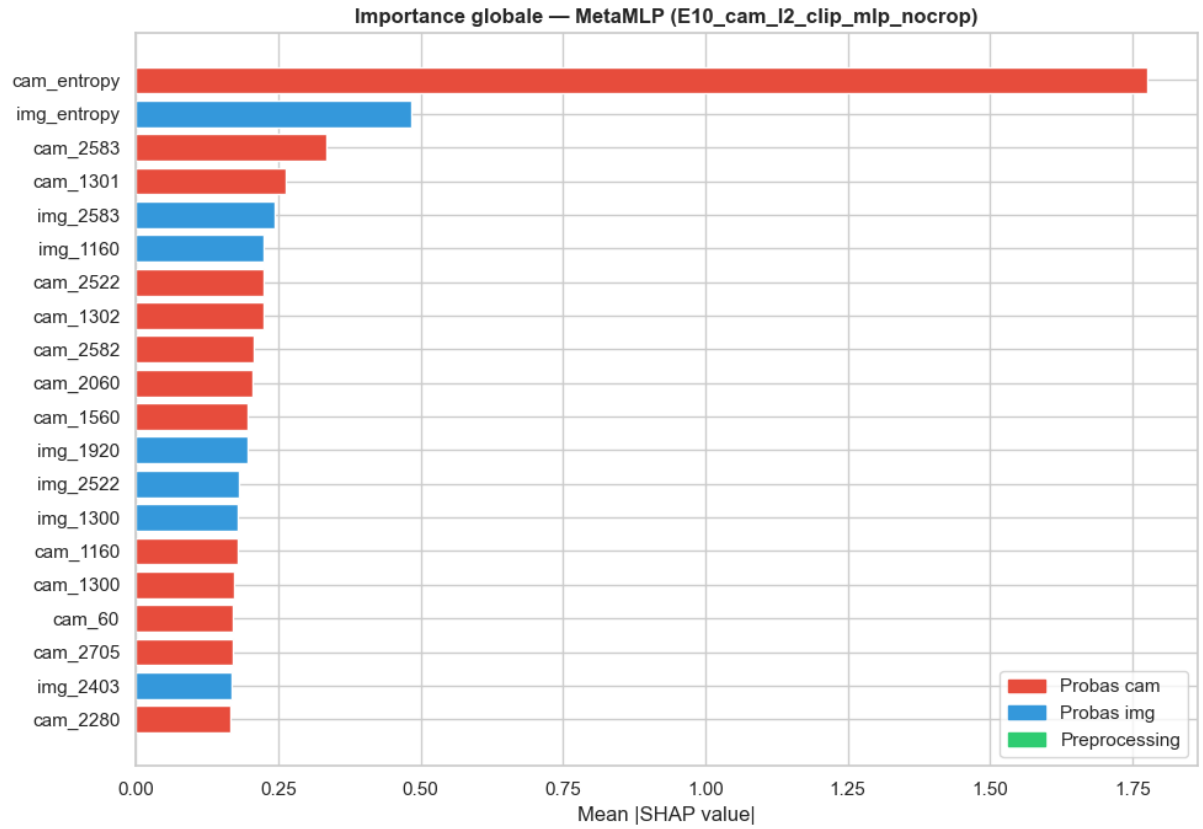


Figure 18 — Importance globale (mean |SHAP value|) des 20 features les plus contributives au MetaMLP E10. cam_entropy domine massivement, suivie loin derrière par img_entropy ; le reste du top 20 est dominé par les probabilités CamemBERT par classe.

Trois enseignements ressortent. Premièrement, l'entropie texte est de très loin la feature la plus importante, environ trois fois et demie supérieure à l'entropie image et près de cinq fois supérieure à la première probabilité brute. Cette domination valide a posteriori le choix d'inclure les méta-scalaires dans le vecteur de stacking. C'est précisément l'incertitude de CamemBERT, et non ses probabilités brutes, qui sert au MetaMLP à arbitrer entre la branche texte et la branche image. Quand CamemBERT est très confiant (entropie faible), le méta-modèle valide sa prédiction ; quand il hésite (entropie élevée), il s'appuie davantage sur l'image et sur l'accord cross-modal. Cette logique de pondération adaptative, formulée comme hypothèse centrale en Étape 1, est ainsi vérifiée quantitativement.

Deuxièmement, les probabilités CamemBERT dominent le top 20. Cette asymétrie reproduit fidèlement l'écart de performance unimodale, plus une branche est performante isolément,

plus le méta-MLP s'appuie sur ses sorties brutes. Les probabilités CamemBERT les plus contributives correspondent à des classes spécifiques pour lesquelles CamemBERT tranche nettement et de manière discriminante.

Troisièmement, les probabilités image apparaissent sur des classes différentes. Quelques classes apparaissent dans les deux modalités (2583, 2522), ce sont des cas où texte et image apportent un signal indépendant et redondant. Mais sur img_1160 ou img_1300, l'image apporte un signal que le texte ne capture pas pleinement, typiquement quand la désignation est courte ou ambiguë et que le visuel lève la décision. C'est exactement le scénario pour lequel le stacking multimodal a été conçu.

5.4 Synthese interpretabilite

Premierement, le texte est plus interprétable et plus fiable : les tokens TF-IDF discriminants sont directement audit-ables par un opérateur métier, ce qui facilitera la mise en production. Deuxiemement, l'image apporte un signal complementaire mais sa contribution est moins interpretable et plafonnee par le bruit visuel intrinseque du dataset (fond blanc, isomorphisme inter-classes). Troisièmement, les meta-features sémantiques (entropie, marge, accord cross-modal) révèlent quand le modèle est incertain et fournissent un score de confiance directement exploitable en production pour décider d'un workflow de validation humaine.

6. Conclusions scientifiques et métiers

6.1 Bilan de la modelisation

Ce projet valide l'approche incrémentale choisie : chaque étape a permis d'améliorer les performances. Le meilleur modèle est le stacking multimodal final (E10), qui combine du texte (CamemBERT), des images (CLIP) et des métadonnées pour atteindre un score F1-weighted de 0,8963, se rapprochant ainsi du vainqueur du challenge SIGIR 2020.

Le passage d'une baseline simple (TF-IDF + LightGBM) au stacking final a permis de **réduire le taux d'erreur de près de 33 %**. Cela prouve l'intérêt concret d'associer le texte et l'image pour ce catalogue de produits.

Systeme	F1 Test	Ecart vs baseline SIGIR	Type
SIGIR 2020 — 1er place du challenge	0,9194	+0,0489	Reference externe
SIGIR 2020 — Baseline officielle	0,8705	—	Reference externe
Notre nouveau meilleur — E10 (cam+CLIP+meta)	**0,8963**	**+0,0258**	**Notre modele**
Stacking E11 (cam+DINOv2 crop+meta)	**0,8948**	**+0,0243**	**Notre modele**
Ancien stacking final E7	0,8879	+0,0174	Notre modele
CamemBERT FT seul (texte)	**0,8717**	**+0,0012**	**Notre modele**
Texte T4 (TF-IDF + e5 + MLP)	0,8632	−0,0073	Notre modele
Baseline TF-IDF + LightGBM (NB2)	0,8321	−0,0384	Notre modele
Image seule (NB4 I2)	0,7486	−0,1219	Notre modele

6.2 Enseignements scientifiques

Cette campagne expérimentale met en lumière plusieurs enseignements scientifiques majeurs. Tout d'abord, sur un catalogue de type e-commerce, la modalité textuelle domine largement la modalité visuelle car le vocabulaire des fiches produits est intrinsèquement plus discriminant que des images souvent génériques.

De plus, la qualité du pré-entraînement s'avère plus décisive que l'adaptation supervisée locale. Des modèles massifs figés comme CLIP ou DINOv2 surpassent un ResNet50 entièrement réentraîné, l'espace sémantique de CLIP étant particulièrement performant car il est déjà aligné sur des concepts textuels.

Enfin, le succès du stacking repose sur sa capacité à exploiter les incertitudes croisées, notamment à travers le signal binaire d'accord entre le texte et l'image. Ce mécanisme, enrichi par des méta-features géométriques simples comme l'entropie ou la marge de confiance, permet au méta-classifieur de calibrer efficacement ses décisions, tandis que les indicateurs de qualité d'image comme le flou ou le zoom n'apportent qu'un gain secondaire.

6.3 Analyse critique

Ce qui a bien fonctionne

La séparation OOF rigoureuse en 5-fold s'est révélée décisive, elle garantit l'absence de fuite d'information entre niveaux du stacking et fournit un benchmark interne fiable. Le caching systématique des embeddings (e5-large, DINOv2, CLIP) a permis près de cinquante itérations en quelques heures, sans devoir recalculer les représentations à chaque essai. La modélisation incrémentale, une expérience par levier, a facilité le débogage et l'isolation des gains. Enfin, l'usage de classe weights balance des la baseline, conformément à la décision prise en Etape 1, a évité le collapse vers les classes majoritaires malgré le déséquilibre 13:1.

Ce qui n'a pas fonctionné comme espéré

Trois choses méritent d'être signalées. Le fine-tuning complet de ResNet50 termine sous DINOv2 en frozen, ce qui constitue un retour négatif sur un investissement compute important. Les images auto-croupées dans le stacking final (experience E8) degradent légèrement le score par rapport au non-croppe (E7), révélant que le crop agressif perd parfois du contexte utile. LightGBM applique aux features stacking (E5) sous-performe nettement LogReg et MLP, la partition par seuil des arbres est moins efficace que la combinaison linéaire suivie d'activations non-lineaires.

Biais et limites du modele

Trois sources de biais résiduels méritent d'etre signalées. D'une part, les classes rares (par exemple Jeux video, ~1 %) sont moins représentées en train, ce qui peut entrainer un sur-apprentissage des classes fréquentés malgre le class_weighting balanced. D'autre part, le corpus est dominé par le français a 84,2 % selon les comptages de l'Etape 1, la performance sur les produits rédigées majoritairement en allemand ou en espagnol n'a pas été évaluée séparément et pourrait s'avérer inférieure à la performance moyenne. Enfin, les modèles pre-entraines (e5-large, DINOv2, CLIP) portent les biais de leurs corpus d'entraînement, ce qui peut affecter certaines catégories culturelles marginales du catalogue Rakuten.

6.4 Perspectives d'amelioration

Pour franchir le dernier palier et égaler le score d'élite du challenge, plusieurs axes de développement sont envisagés. La première priorité consiste à réaliser un fine-tuning complet

et simultané des branches CamemBERT et CLIP directement au sein du processus de validation croisée. Pour enrichir le signal, nous prévoyons également d'introduire des techniques de Test-Time Augmentation sur les images, ainsi que de la traduction aller-retour pour densifier le texte des catégories les plus rares. Par ailleurs, l'optimisation des hyperparamètres du méta-classifieur final via un Grid Search systématique permettra d'ajuster finement la régularisation et l'architecture du MLP, garantissant ainsi la meilleure combinaison possible des prédictions. Enfin, la transition vers un environnement de production impliquera l'usage de la distillation de connaissances afin de transférer la puissance de notre stacking lourd vers un modèle unique et rapide, complété par une phase de calibration de Platt pour sécuriser la fiabilité des probabilités finales.

6.5 Implications metier pour Rakuten

Trois implications opérationnelles directes peuvent être tirées de ces résultats pour le métier Rakuten. Premièrement, le pipeline atteint un F1-weighted supérieur à 0,88 sur le test, niveau compatible avec un déploiement en production en assistance des vendeurs. Lors de la création d'un nouveau produit, le système peut proposer automatiquement la catégorie en s'appuyant sur la désignation, la description et l'image fournies. Deuxièmement, le score de confiance produit par le meta-MLP (max_confidence) permet de définir un seuil de routage opérationnel, les produits a confiance inferieure a 0,5, environ 7 % du flux, peuvent être routés vers une verification humaine, tandis que les 93 % restants sont catégorisés automatiquement avec un taux d'erreur estimé inférieur à 5 %. Cette architecture en pipeline confiance-driven optimise simultanément la qualité et le coût opérationnel. Troisièmement, le modèle texte seul (NB3 T4, F1 = 0,8632) est suffisant pour les flux ou l'image n'est pas encore disponible (création initiale d'un produit), évitant un blocage de l'experience utilisateur dans l'attente du téléversement de la photo.

Annexes

Annexe A - Recapitulatif des Paramètres et Hyperparamètres

Modele	Hyperparametre	Valeur
Baseline NB2	TF-IDF max_features	50 000
Baseline NB2	TF-IDF ngram_range	(1, 2)
Baseline NB2	TF-IDF sublinear_tf	True
Baseline NB2	LightGBM num_leaves	31
Baseline NB2	LightGBM learning_rate	0,1
Baseline NB2	Early stopping patience	20
NB3 - T4 MLP texte	Hidden dims	(1024 vers 512 vers 256)
NB3 - T4 MLP texte	Dropout	0,3
NB3 - T4 MLP texte	Optimiseur	Adam, lr = 1e-3
NB3 - T4 MLP texte	Folds OOF	5 stratifies
NB4 - DINOv2	Backbone	facebook/dinov2-base (768-d)
NB4 - CLIP	Backbone	ViT-B/32 LAION-2B (512-d)
NB4 - ResNet50	Phases unfreezing	3 (5+3+2 epochs)
NB4 - ResNet50	Learning rates	1e-4 vers 1e-5 vers 1e-6
NB4 - MixUp DINOv2 / CLIP	alpha	0,2
NB5 - MetaMLP	Architecture	Linear(71 vers 256 vers 128 vers 64 vers 27)
NB5 - MetaMLP	BatchNorm + Dropout	0,3
NB5 - MetaMLP	Early stopping patience	20 sur F1 val
NB5 - MetaMLP	Optimiseur	Adam, lr = 1e-3, wd = 1e-4
NB5 - MetaMLP	Batch size	256

Annexe B - Artefacts produits

Les modeles entraines et leurs metadonnees sont sauvegardes dans models/clean/ et serviront notamment a l'application Streamlit de demonstration developpee dans src/streamlit/.

Fichier	Description	Taille approx.
baseline_lgbm.pkl	Modele baseline NB2 (LightGBM)	~10 Mo
tfidf_vectorizer.pkl	Vectorizer TF-IDF 50k features	~5 Mo
ctfidf_centroids.npy	27 centroides c-TF-IDF	~10 Mo
resnet50_ft.pth	ResNet50 fine-tuned (state dict)	~94 Mo
dino_ft_best.pth	DINOv2 fine-tuned dernier bloc	~340 Mo
clip_ft_best.pth	CLIP ViT-B/32 fine-tuned	~600 Mo
meta_E7_meta_prep_mlp.pkl	Meta-MLP final E7 (71 vers 27)	~1 Mo
meta_best.pkl	Lien vers le meilleur meta-modele	~1 Mo
label_encoder.pkl	LabelEncoder 27 classes	<1 Ko
benchmark_final.csv	Tableau resultats vs SIGIR	<1 Ko
logreg_surrogate.pkl	Surrogate LogReg interpretabilite	~50 Mo

Annexe C - Pipeline de notebooks

Le pipeline est entierement reproductible via la sequence de cinq notebooks suivants, contenus dans notebooks/main_notebooks/. Chaque notebook depend du precedent via les artefacts deposes dans data/processed_clean/ et models/clean/.

- 01_preprocessing_eda.ipynb : nettoyage, EDA, splits stratifies, exports features (continuite directe avec l'Etape 1).
- 02_baseline_multimodale.ipynb : TF-IDF + features image + LightGBM. F1 test = 0,8321.
- 03_experiments_texte.ipynb : c-TF-IDF, multilingual-e5-large, fusion, OOF 5-fold. F1 test = 0,8632.
- 04_experiments_images.ipynb : DINOv2, CLIP, ResNet50, GradCAM, OOF 5-fold. F1 test = 0,7486.
- 05_stacking_final.ipynb : assemblage des OOF, meta-features, MetaMLP, benchmark final. F1 test = 0,8879.

Une application Streamlit (src/streamlit/app.py + pipeline.py) consomme les artefacts produits pour fournir une demonstration interactive temps reel sur produit unique : l'utilisateur saisit une designation, une description optionnelle et une image, et le pipeline renvoie la classe predite avec son score de confiance, accompagne d'une visualisation GradCAM sur l'image.

Annexe D - Reproductibilite et environnement

Parametre	Valeur
SEED global	42 (numpy, torch, sklearn)
Split train / val / test	Stratifie 70 / 15 / 15 (continuite Etape 1)
Folds OOF stacking	5 stratifies sur la classe

Tailles des splits	59 441 / 12 737 / 12 738
Metrique principale	F1-score weighted (sklearn)
Metrique secondaire	F1-score macro (surveillance classes rares)
Python	3.10
PyTorch	2.x avec CUDA
scikit-learn	1.x
GPU utilise	NVIDIA RTX (training fine-tuning et embeddings)

Annexe E - Expériences Ablation sur l'input

Texte : Objectif et protocole

Cette série d'expériences teste l'impact du choix d'input textuel sur les performances de classification, en maintenant un pipeline fixe : TF-IDF (50 000 features, bigrammes, sublinear_tf=True, min_df=2) + LightGBM (1 000 estimateurs, learning_rate=0.05, num_leaves=127, early stopping 50 rounds, poids de classes balancés). Le split 70/15/15

stratifié est identique pour toutes les expériences (seed=42), garantissant la comparabilité directe des scores.

Exp_ID	Input	Traduction
EXP_T01	Désignation seule	Non
EXP_T02	Description seule (fallback désignation)	Non
EXP_T03	Désignation + Description combinées	Non
EXP_T03b	Désig. + Desc. + Stemming SnowballFR + Stopwords	Non
EXP_T04	Désignation seule	→ Français (GoogleTranslator)
EXP_T05	Désignation + Description combinées	→ Français
EXP_T_KW	c-TF-IDF (Chi², k=20k) + similarité cosinus centroides	Non

EXP_T01 — Désignation seule

Objectif : Établir une baseline minimale en utilisant uniquement le champ designation, court et dense en termes-clés, afin de mesurer sa contribution isolée.

Avec seulement 6 926 features effectivement utilisées (sur 50 000 possibles), la désignation seule offre une performance solide. La concision du champ — médiane de 11 mots — est compensée par sa densité informationnelle : chaque désignation contient le type de produit, la marque et les caractéristiques principales. Ce score constitue le plancher de référence pour toutes les expériences suivantes.

Résultats : Weighted F1 = 0,7899 | Macro F1 = 0,7639 | Accuracy = 0,7863 | Best iter = 114 | Durée = 12 min

EXP_T02 — Description seule (fallback désignation)

Objectif : Évaluer si la description, plus longue et plus détaillée, constitue un meilleur signal que la désignation.

L'input est description_clean quand disponible ; pour les 35,3 % de produits sans description (confirmé en Étape 1 : $\chi^2 = 2\,847{,}4$; $p < 0{,}001$), la désignation sert de fallback. Le vocabulaire effectivement utilisé est 6× plus large qu'EXP_T01 (41 538 features), reflétant la richesse lexicale des descriptions. Le gain par rapport à EXP_T01 est néanmoins marginal

(+0,0028), pour deux raisons : un tiers des produits retombent mécaniquement sur la désignation, et les descriptions très hétérogènes (1 à 2 068 mots) génèrent du bruit. Le meilleur macro F1 (0,7791 vs 0,7639) indique que la description aide sur les classes rares, qui bénéficient d'un texte plus riche. Le surcoût computationnel (69 min vs 12 min) rend ce gain difficilement justifiable en isolation.

Résultats : Weighted F1 = 0,7927 | Macro F1 = 0,7791 | Accuracy = 0,7909 | Best iter = 104 | Durée = 69 min

EXP_T03 — Désignation + Description combinées

Objectif : Tester si la concaténation des deux champs maximise le signal textuel disponible.

Avec 43 253 features effectives, la combinaison des deux champs produit un gain de +0,0375 par rapport à EXP_T01 (+0,0347 vs EXP_T02). Les produits sans description ne sont pas pénalisés : text_combined contient uniquement la désignation pour eux, sans imputation artificielle. Cette configuration devient la référence interne pour les expériences suivantes.

Résultats : Weighted F1 = 0,8274 | Macro F1 = 0,8114 | Accuracy = 0,8250 | Best iter = 106 | Durée = 84 min

EXP_T03b — Désignation + Description + Stemming + Stopwords

Objectif : Tester si la normalisation lexicale (stemming SnowballFR + stopwords multilingues) réduit le bruit et améliore EXP_T03.

Le stemming SnowballStemmer français ramène chaque token à sa racine (vélos → vélo, piscines → piscin) et les stopwords NLTK (français + anglais + allemand + espagnol) sont supprimés. La réduction du vocabulaire est notable : 28 612 features effectives contre 43 253 pour EXP_T03. Le résultat est contre-intuitif : le stemming dégrade les performances de −0,0126 par rapport à EXP_T03. L'explication tient à la nature du corpus e-commerce : les noms propres, marques et références techniques (Nintendo, ResNet, A4) sont sur-stemmés ou supprimés comme stopwords, perdant précisément les tokens les plus discriminants par classe. Cette expérience confirme que la normalisation lexicale agressive est inadaptée aux catalogues produits.

Résultats : Weighted F1 = 0,8148 | Macro F1 = 0,7970 | Accuracy = 0,8117 | Best iter = 104 | Durée = 45 min

EXP_T04 — Désignation seule traduite en français

Objectif : Tester si l'homogénéisation linguistique améliore la classification sur la désignation seule.

La langue est détectée via langdetect (seed=42) ; les 25,4 % de désignations non-françaises sont traduites via GoogleTranslator (deep-translator), avec cache local pour éviter les re-traductions. La traduction dégrade les performances de −0,0112 par rapport à EXP_T01. Trois mécanismes l'expliquent : (i) les erreurs de traduction sur les marques et références ("camion man" → "camion homme") détruisent des tokens hautement discriminants ; (ii) la diversité linguistique constitue un signal de classe indirect — les livres allemands, les jeux japonais importés sont fortement corrélés à leur langue d'origine, signal effacé par la traduction ; (iii) l'alignement cross-langue est déjà géré implicitement par le TF-IDF (des tokens anglais ou allemands récurrents dans une classe sont naturellement discriminants). Cette expérience confirme la recommandation formulée en Étape 1 : ne pas traduire pour les modèles bag-of-words.

Résultats : Weighted F1 = 0,7787 | Macro F1 = 0,7524 | Accuracy = 0,7744 | Best iter = 112 | Durée = 11 min

EXP_T05 — Désignation + Description combinées → traduites en français

Objectif : Tester si la traduction du texte combiné améliore EXP_T03.

18,0 % des textes combinés sont détectés comme non-français et traduits (cache text_combined_translated.csv). Le résultat est cohérent avec EXP_T04 : la traduction dégrade également la configuration combinée (−0,0045 vs EXP_T03). Combinée à la masse de descriptions françaises présentes dans text_combined, la traduction introduit plus de bruit qu'elle n'en supprime.

Résultats : Weighted F1 = 0,8229 | Macro F1 = 0,8057 | Accuracy = 0,8201 | Best iter = 107 | Durée = 68 min

EXP_T_KW — c-TF-IDF + sélection Chi² + similarité cosinus

Objectif : Exploiter la structure inter-classes du corpus via les centroïdes c-TF-IDF pour enrichir la représentation TF-IDF classique.

L'approche combine deux sources de signal : (1) TF-IDF réduit à 20 000 features par sélection Chi² (les tokens les plus discriminants statistiquement) ; (2) 27 features de similarité cosinus entre chaque document et les centroïdes c-TF-IDF de chaque classe — vecteurs denses construits en agrégeant les produits d'une même classe puis normalisés L2.

Cette représentation hybride (20 027 dimensions) apporte au classifieur un signal semi-supervisé : non seulement quels mots sont présents, mais à quelle classe le texte ressemble lexicalement. Le gain est de +0,0047 par rapport à EXP_T03 (+0,0422 vs EXP_T01), avec un temps d'entraînement réduit (39 min) grâce à la sélection Chi². Ce pipeline constitue la meilleure configuration texte TF-IDF + LightGBM de cette série.

Résultats : Weighted F1 = 0,8321 | Macro F1 = 0,8176 | Accuracy = 0,8317 | Best iter = 77 | Durée = 39 min

Tableau de synthèse

Exp_ID	Description	W-F1	M-F1	Δ vs T01
EXP_T04	Désig. traduite	0,7787	0,7524	−0,0112
EXP_T01	Désig. seule	0,7899	0,7639	—
EXP_T02	Desc. seule + fallback	0,7927	0,7791	+0,0028
EXP_T05	Combiné traduit	0,8229	0,8057	+0,0330
EXP_T03b	Combiné + stemming	0,8148	0,7970	+0,0249
EXP_T03	Combiné	0,8274	0,8114	+0,0375
EXP_T_KW	c-TF-IDF + Chi²	0,8321	0,8176	+0,0422

Quatre enseignements ressortent de cette ablation:

(1) La concaténation désignation + description est systématiquement supérieure à l'usage isolé de l'un ou l'autre champ.

(2) La traduction automatique dégrade les performances quelle que soit la configuration, confirmant que la diversité linguistique est un signal de classe plutôt qu'un bruit.

(3) Le stemming agressif nuit sur un corpus e-commerce dominé par les noms propres et les références techniques.

(4) L'enrichissement par c-TF-IDF apporte un gain marginal mais reproductible (+0,0047 vs EXP_T03), validant l'hypothèse que la structure inter-classes du corpus est exploitable comme feature.

Expériences image — Ablation ResNet50

Objectif et protocole

Cette série d'expériences teste l'impact des choix d'architecture et d'entraînement sur les performances de classification image, en maintenant un backbone ResNet50 (pré-entraîné ImageNet) et en faisant varier une seule variable à la fois. La tête FC est remplacée par Dropout(0.3) + Linear(2048, 27). L'optimiseur est AdamW avec scheduler CosineAnnealingLR sur 10 epochs (GPU Kaggle T4 16 Go). Le split 80/20 stratifié est identique pour toutes les expériences (seed=42).

Exp_ID	Autocrop	Entraînement	Régularisation
EXP_I01	Non	Fine-tuning complet	Dropout=0.3
EXP_I02	Oui	Fine-tuning complet	Dropout=0.3
EXP_I03	Non	Tête seule (backbone gelé)	Dropout=0.3
EXP_I04	Non	Unfreeze progressif (layer4 → tout)	Dropout=0.3
EXP_I05	Non	Fine-tuning complet, StepLR	Dropout=0.3
EXP_I06	Oui	Fine-tuning complet + augmentation	Dropout=0.5

EXP_I01 — Fine-tuning complet, sans autocrop (référence)

Objectif : Établir la baseline ResNet50 en fine-tunant l'ensemble du backbone sans preprocessing particulier.

L'ensemble du réseau est dégelé dès la première epoch. Le scheduler CosineAnnealingLR ramène η à η_min=1e-6 sur 10 cycles, limitant la perturbation des couches basses. Le class_weight='balanced' corrige le déséquilibre 13:1. Ce score constitue la référence image de l'ablation.

Résultats : Weighted F1 = 0,6774 | Durée ≈ 2h (GPU T4)

EXP_I02 — Fine-tuning complet, avec autocrop

Objectif : Mesurer l'impact de la fonction auto_crop() développée en Étape 1 sur les performances ResNet50.

L'autocrop détecte la bounding box du produit sur fond blanc et recadre avec une marge de 2 %. Contrairement à l'hypothèse, l'autocrop dégrade légèrement les performances (−0,0043 vs EXP_I01). Pour les architectures entraînées bout-en-bout, le fond blanc contient un signal

de contexte indirect exploitable — que l'autocrop supprime. Ce résultat est cohérent avec l'expérience E8 du stacking final (NB5), où le crop agressif dégrade également de −0,003.

Résultats : Weighted F1 = 0,6731 | Δ vs I01 = −0,0043

EXP_I03 — Backbone gelé (tête seule entraînée)

Objectif : Tester si un simple transfer learning sans fine-tuning est suffisant.

Seule la tête FC est entraînée (lr=1e-3), le backbone ResNet50 reste gelé. Le score plafonne (0,6476), confirmant que les features ImageNet seules sont insuffisantes pour distinguer des produits catalogue sur fond blanc uniformément blanc. Cette expérience justifie a posteriori le fine-tuning complet pour toutes les expériences suivantes.

Résultats : Weighted F1 = 0,6476 | Δ vs I01 = −0,0298

EXP_I04 — Unfreeze progressif

Objectif : Tester si une stratégie d'unfreeze progressif (couche par couche) dépasse le fine-tuning direct.

Phase 1 (epochs 1-5) : head seule (lr=1e-3). Phase 2 (epochs 6-8) : layer4 dégelé (lr=1e-4). Phase 3 (epochs 9-10) : backbone complet (lr=1e-5). Le résultat (0,6522) dépasse le frozen (+0,0046 vs I03) mais reste inférieur au fine-tuning direct (−0,0252 vs I01). La progression couche-par-couche ne compense pas la perte de mise à jour précoce du backbone.

Résultats : Weighted F1 = 0,6522 | Δ vs I01 = −0,0252

EXP_I05 — Scheduler StepLR

Objectif : Comparer CosineAnnealingLR et StepLR sur la convergence.

Le scheduler StepLR (step_size=3, gamma=0.1) divise lr par 10 toutes les 3 epochs. Le score légèrement inférieur (0,6594 vs 0,6774) confirme que la décroissance progressive en cosinus est mieux adaptée au fine-tuning ResNet50 sur ce corpus.

Résultats : Weighted F1 = 0,6594 | Δ vs I01 = −0,0180

EXP_I06 — Augmentation + Dropout renforcé

Objectif : Tester si une régularisation agressive améliore la généralisation.

Augmentation entraînement : RandomHorizontalFlip, ColorJitter(brightness=0.2, contrast=0.2, saturation=0.2), RandomRotation(15°). Dropout porté à 0,5. Autocrop activé. Le score (0,6650) est inférieur à EXP_I01 (−0,0124) : la régularisation supplémentaire nuit ici, car les images catalogue sont standardisées (fond blanc uniforme, prise de vue contrôlée) et peu sujettes à la variance d'illumination ou de rotation.

Résultats : Weighted F1 = 0,6650 | Δ vs I01 = −0,0124

Tableau de synthèse

Exp_ID	Description	W-F1	Δ vs I01
EXP_I03	Backbone gelé	0,6476	−0,0298
EXP_I04	Unfreeze progressif	0,6522	−0,0252
EXP_I05	Fine-tuning + StepLR	0,6594	−0,0180
EXP_I06	Fine-tuning + augmentation + Dropout=0,5	0,6650	−0,0124
EXP_I02	Fine-tuning + autocrop	0,6731	−0,0043
EXP_I01	Fine-tuning complet (référence)	0,6774	—

Quatre enseignements ressortent de cette ablation image :

(1) Le fine-tuning complet dès la première epoch est la stratégie optimale pour ResNet50 sur ce corpus de taille modérée.

(2) L'autocrop dégrade les performances en fine-tuning bout-en-bout : le fond blanc contient un signal de contexte implicitement exploité par le backbone — observation cohérente avec E8 vs E7 dans le stacking (chapitre 4).

(3) La régularisation agressive (augmentation forte + dropout élevé) est contre-productive sur les images catalogue standardisées.

(4) Le plafond ResNet50 (F1 ≈ 0,68) est nettement inférieur à CLIP frozen (NB4 I2, F1 = 0,7486) : la qualité du pré-entraînement contrastif sur 2 milliards de paires CLIP compte plus que la capacité d'adaptation supervisée sur ce corpus de 60 000 images.

Bibliographie

Datasets et challenges

[1] Rakuten France (2020). Rakuten France Multimodal Product Data Classification - SIGIR eCommerce 2020 Data Challenge. https://sigir-ecom.github.io/ecom2020/

[2] ENS Data Challenge. Rakuten France Multimodal Product Data Classification. https://challengedata.ens.fr/challenges/35

[3] SIGIR eCommerce Workshop (2020). Proceedings & Leaderboard. https://sigir-ecom.github.io/data-task.html

[4] Communaute aventuresdata.com (2020). Nomenclature des prdtypecode Rakuten. https://aventuresdata.com/rakuten/rakuten_desc.html

Modeles texte

[5] Wang, L. et al. (2024). Multilingual E5 Text Embeddings: A Technical Report. arXiv:2402.05672. Modele utilise : intfloat/multilingual-e5-large. https://huggingface.co/intfloat/multilingual-e5-large

[6] Reimers, N. & Gurevych, I. (2019). Sentence-BERT: Sentence Embeddings using Siamese BERT-Networks. EMNLP. Bibliotheque sentence-transformers, famille a laquelle appartiennent multilingual-e5-large et paraphrase-multilingual-mpnet-base-v2 envisage en Etape 1.

[7] Grootendorst, M. (2022). BERTopic: Neural topic modeling with a class-based TF-IDF procedure. arXiv:2203.05794. Methode c-TF-IDF utilisee pour generer les 27 features de similarite par classe.

[8] Ke, G., Meng, Q., Finley, T., et al. (2017). LightGBM: A Highly Efficient Gradient Boosting Decision Tree. NeurIPS. https://lightgbm.readthedocs.io/

[9] Salton, G. & Buckley, C. (1988). Term-weighting approaches in automatic text retrieval. Information Processing & Management, 24(5). Methode TF-IDF utilisee comme baseline.

[10] Lim, J. & Conneau, A. et al. CamemBERT, mBERT - mentionnes comme pistes en Etape 1, retenus dans la liste des perspectives d'amelioration (chapitre 6.4).

Modeles vision

[11] Oquab, M., Darcet, T., Moutakanni, T., et al. (2024). DINOv2: Learning Robust Visual Features without Supervision. arXiv:2304.07193. Modele facebook/dinov2-base utilise en frozen feature extractor et fine-tuning.

[12] Radford, A., Kim, J. W., Hallacy, C., et al. (2021). Learning Transferable Visual Models From Natural Language Supervision (CLIP). ICML. Backbone CLIP ViT-B/32 (LAION-2B-s34B-b79K) via OpenCLIP.

[13] He, K., Zhang, X., Ren, S. & Sun, J. (2016). Deep Residual Learning for Image Recognition (ResNet). CVPR. ResNet50 pre-entraine ImageNet-1K utilise en fine-tuning progressif.

[14] Schuhmann, C., Beaumont, R., Vencu, R., et al. (2022). LAION-5B: An open large-scale dataset for training next generation image-text models. NeurIPS.

[15] Zhang, H., Cisse, M., Dauphin, Y. & Lopez-Paz, D. (2018). mixup: Beyond Empirical Risk Minimization. ICLR. Regularisation MixUp (alpha = 0,2) utilisee lors du fine-tuning DINOv2 et CLIP.

[16] Caron, M., Touvron, H., Misra, I., et al. (2021). Emerging Properties in Self-Supervised Vision Transformers (DINO). ICCV. Inspiration des cartes d'attention CLS utilisees sur DINOv2.

Interpretabilite

[17] Selvaraju, R. R., Cogswell, M., Das, A., et al. (2017). Grad-CAM: Visual Explanations from Deep Networks via Gradient-based Localization. ICCV. Methode utilisee pour visualiser les zones discriminantes du ResNet50 et CLIP.

[18] Breiman, L. (2001). Random Forests / Permutation Importance. Machine Learning, 45. Methode utilisee pour mesurer la contribution des meta-features (E6 vs E7).

[19] Molnar, C. (2022). Interpretable Machine Learning. Reference generale sur les surrogate models et le permutation importance. https://christophm.github.io/interpretable-ml-book/

Methodes d'ensemble et stacking

[20] Wolpert, D. H. (1992). Stacked Generalization. Neural Networks, 5(2). Article fondateur du stacking, methode utilisee pour la fusion multimodale finale (E3 a E8).

[21] Hastie, T., Tibshirani, R. & Friedman, J. (2009). The Elements of Statistical Learning. Springer. Reference sur les methodes d'ensemble, la regularisation et la cross-validation.

[22] Pedregosa, F. et al. (2011). Scikit-learn: Machine Learning in Python. JMLR. Utilise pour LogisticRegression, StratifiedKFold, classification_report, F1-weighted, permutation_importance.

Outils et frameworks

[23] Paszke, A. et al. (2019). PyTorch: An Imperative Style, High-Performance Deep Learning Library. NeurIPS. Framework utilise pour tous les modeles deep learning.

[24] Wolf, T. et al. (2020). Transformers: State-of-the-Art Natural Language Processing. EMNLP Demos. Hugging Face Transformers, utilise pour multilingual-e5-large.

[25] Ilharco, G. et al. (2021). OpenCLIP. https://github.com/mlfoundations/open_clip. Implementation utilisee pour CLIP ViT-B/32 LAION.

[26] Streamlit Inc. (2024). Streamlit: The fastest way to build data apps. https://streamlit.io. Utilise pour l'application de demonstration.

Donnees et preprocessing (continuite Etape 1)

[27] py3langid (fork de Lui & Baldwin 2012). Detection automatique de langue, utilisee en Etape 1 pour produire la feature language.

[28] Bird, S., Klein, E. & Loper, E. (2009). Natural Language Processing with Python (NLTK). O'Reilly. Stop-words multilingues et stemmer SnowballFR appliques au pipeline text_combined de l'Etape 1.

[29] Bradski, G. (2000). The OpenCV Library. Dr. Dobb's. Utilise pour le calcul de la variance Laplacien (blur_score) et pour la fonction auto_crop() developpee en Etape 1.

[30] Honnibal, M. & Montani, I. (2017). spaCy 3: Industrial-strength NLP. Utilise pour la tokenisation et le filtrage lemmatique.

[31] Pandas Development Team (2024). pandas: powerful Python data analysis toolkit. https://pandas.pydata.org/

[32] Harris, C. R. et al. (2020). Array programming with NumPy. Nature, 585.

[33] Hunter, J. D. (2007). Matplotlib: A 2D graphics environment + Waskom, M. (2021). Seaborn. JOSS. Bibliotheques de visualisation utilisees pour les figures du rapport et de l'Etape 1.

Documentation interne projet

[34] Veyssiere S., HENRY X., Tingfen Y., Tarik O. (mars 2026). Rapport Etape 1 : Exploration des donnees, datavisualisation et pre-processing. Equipe projet Liora.